Computer Architecture

Lecture 14:

LeakyHammer & IMPACT & CoMeT

Nisa Bostanci

Prof. Onur Mutlu

ETH Zürich

Fall 2025

7 November 2025

SAFARI

ETH zürich

Brief Self Introduction



- **F. Nisa Bostanci**
  - Researcher @ SAFARI Research Group since September 2018
    - PhD @ ETH Zurich (November 2022 – ongoing)
  - nisa.bostanci@safari.ethz.ch
  - https://nisabostanci.github.io/

- Research interests:
  - Computer architecture
  - Memory system security
  - Hardware security, safety, reliability, performance, availability, fairness, energy efficiency
  - Processing-in-memory architectures
  - Hardware/software cooperation
  - …

Cutting-Edge Research Papers in This Talk

- Understanding and Mitigating Covert Channel and Side Channel Vulnerabilities Introduced by RowHammer Defenses, MICRO'25
- Revisiting Main Memory-Based Covert and Side Channel Attacks in the Context of Processing-In-Memory, DSN'25
- CoMeT: Count-Min-Sketch-based Row Tracking to Mitigate RowHammer at Low Cost, HPCA'24
- (If time permits) ABACuS: All-Bank Activation Counters for Scalable and Low Overhead RowHammer Mitigation, USENIX Security'24







Understanding and Mitigating Covert Channel and Side Channel Vulnerabilities Introduced by RowHammer Defenses

F. Nisa Bostancı

O. Canpolat A. Olgun I. E. Yüksel K. Kanellopoulos
M. Sadrosadati A. G. Yağlıkçı O. Mutlu

ETH zürich SAFARI CISPA HELMHOLTZ CENTER FOR INFORMATION SECURITY

Talk Outline

I. Motivation and Problem

II. Key Observation

III. LeakyHammer and Case Studies

IV. Mitigating LeakyHammer

V. Conclusion

Talk Outline

I. **Motivation and Problem**

II. Key Observation

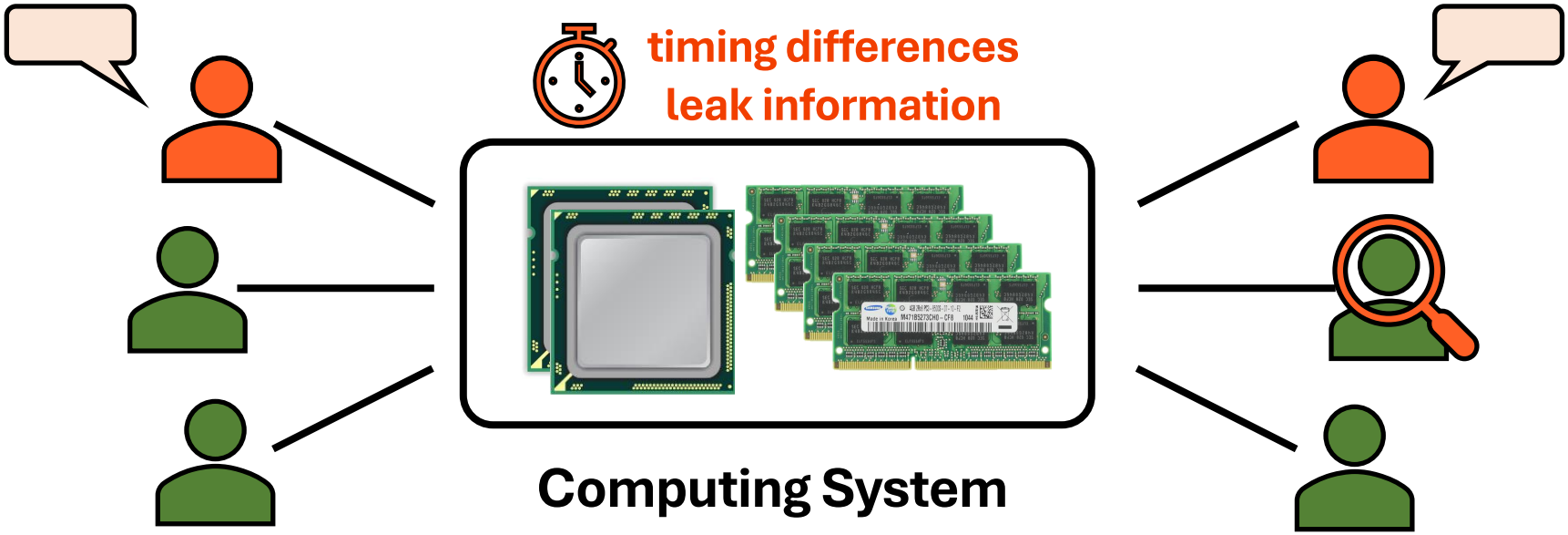
III. LeakyHammer and Case Studies

IV. Mitigating LeakyHammer

V. Conclusion

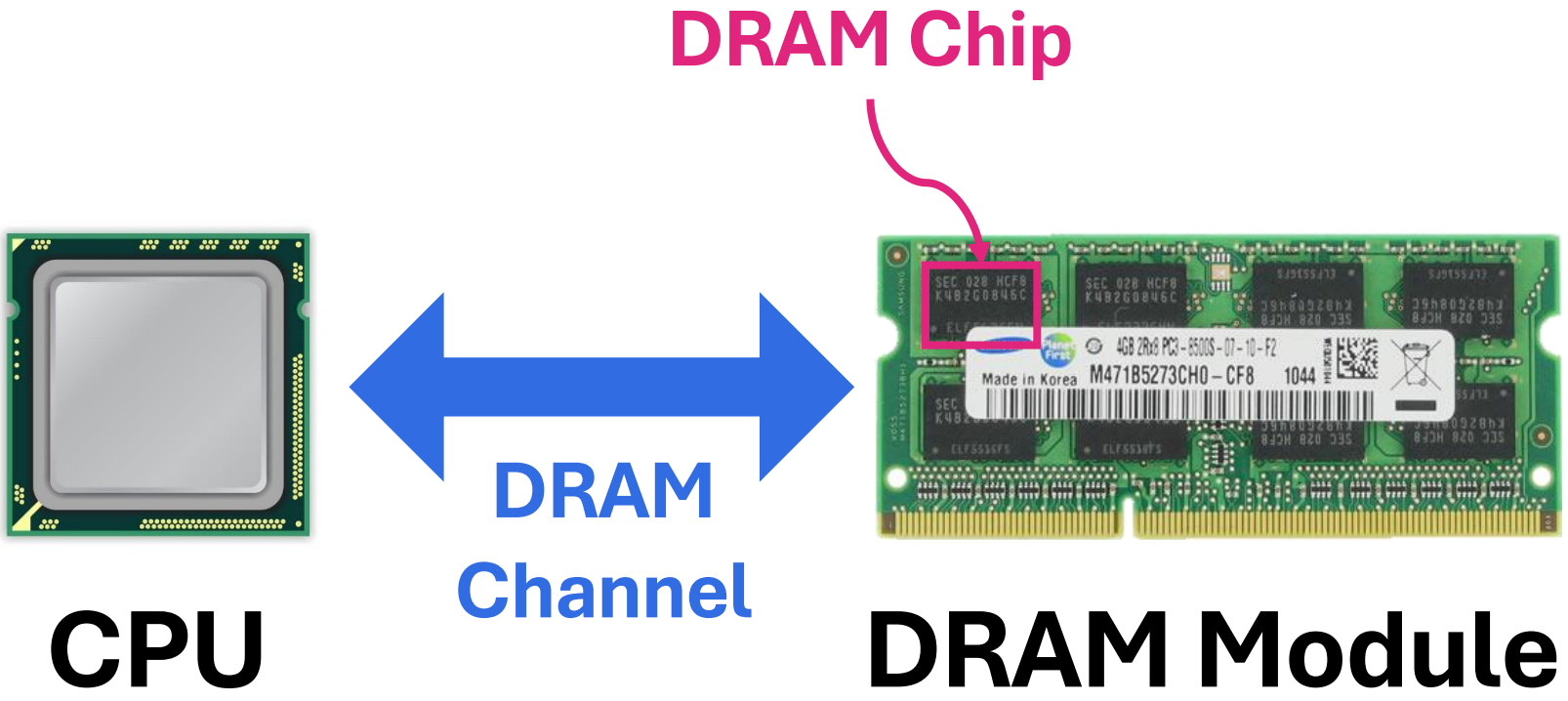
A Typical DRAM-Based Computing System

- Modern systems are vulnerable to security attacks such as covert channel and side channel attacks



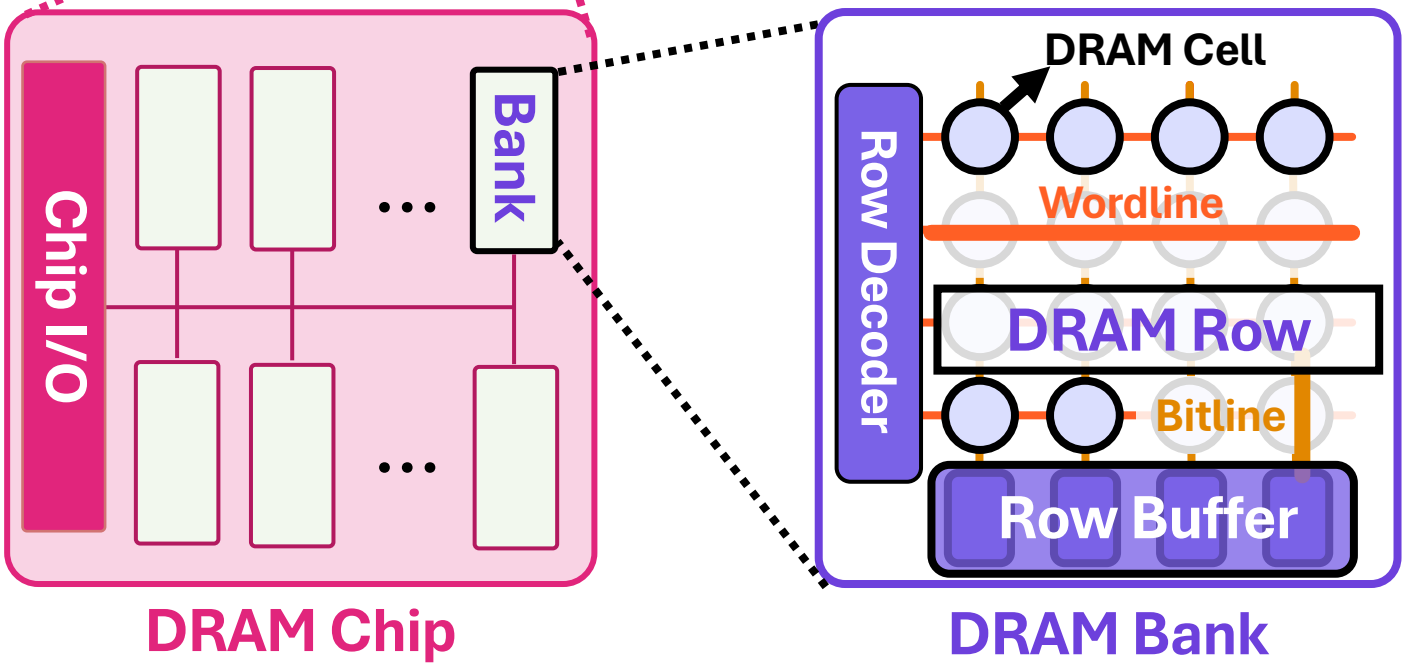
- A set of attacks exploit hardware structures that result in different execution time for a type of operation based on the input or the current state of the structure

A Typical DRAM-Based Computing System



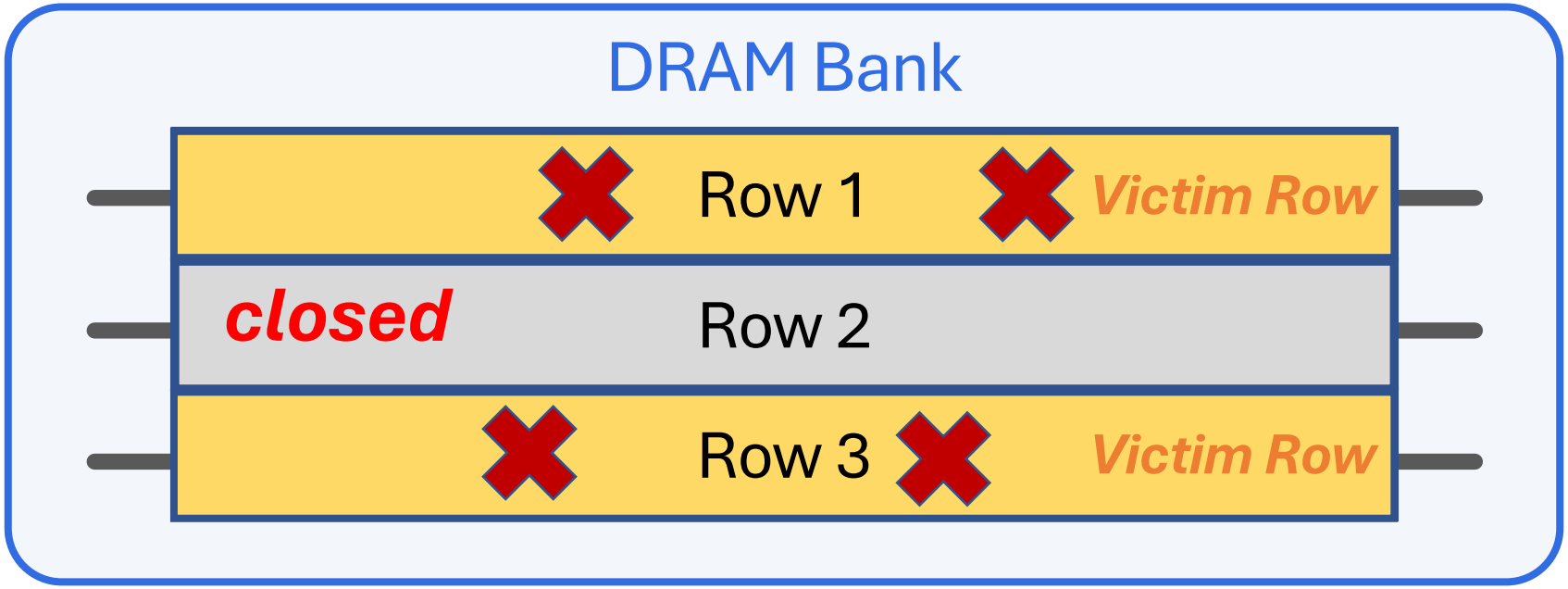
DRAM Organization





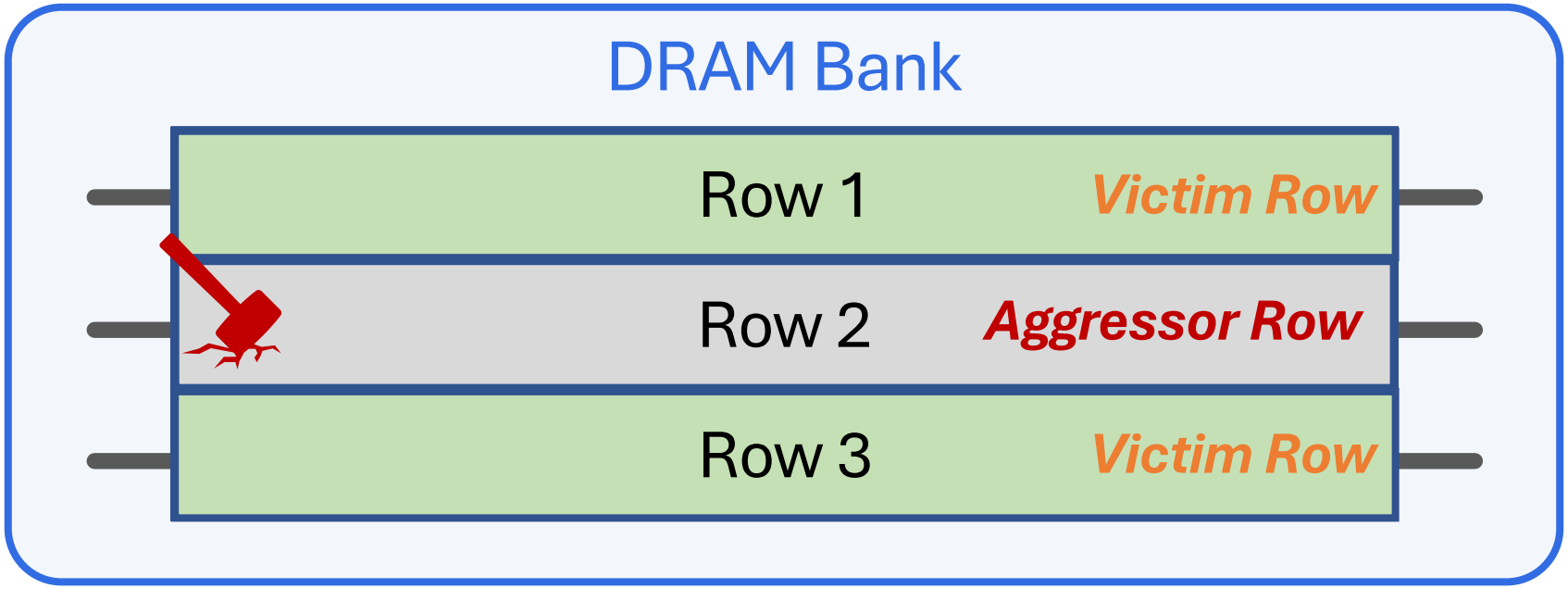
DRAM Read Disturbance

- Read disturbance in DRAM breaks memory isolation
- Prominent example: RowHammer



Repeatedly **opening** (activating) and **closing** (precharging) a DRAM row causes **RowHammer bitflips** in nearby cells

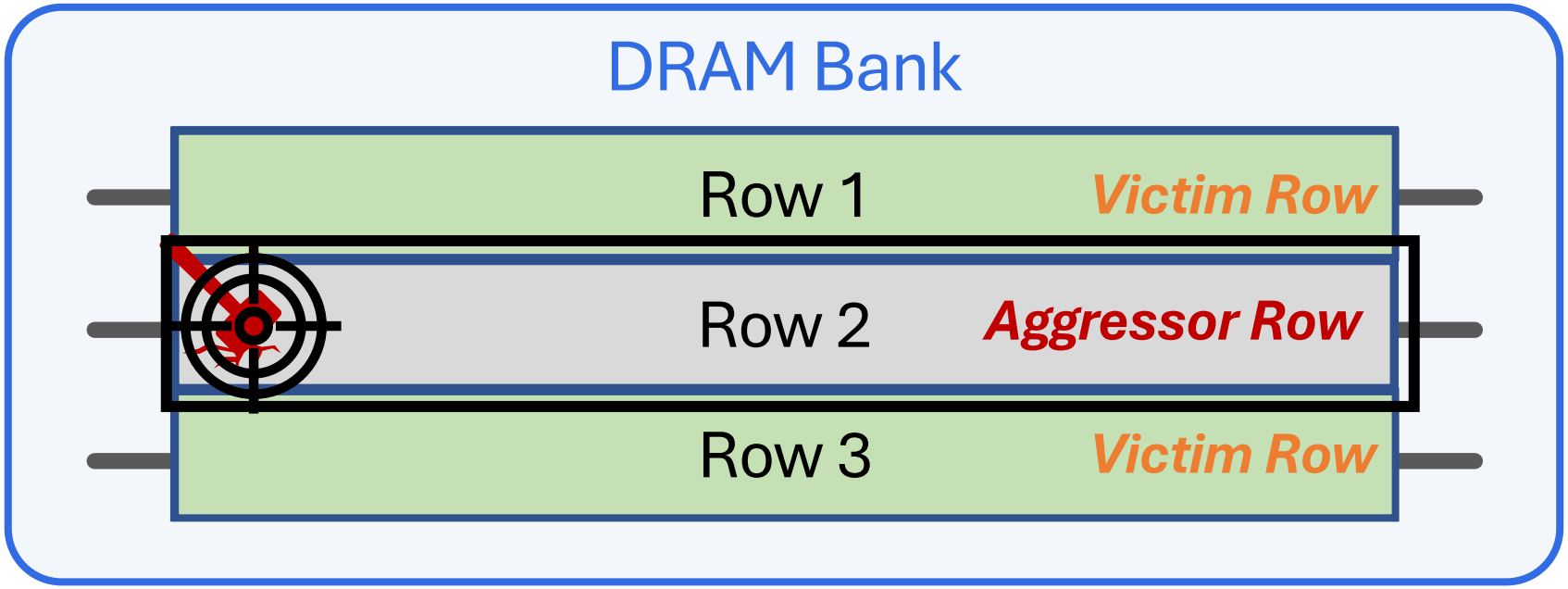
Mitigating Read Disturbance (I): Preventive Refresh



Refreshing potential victim rows
mitigates read disturbance bitflips

[Bostanci+ HPCA'24]

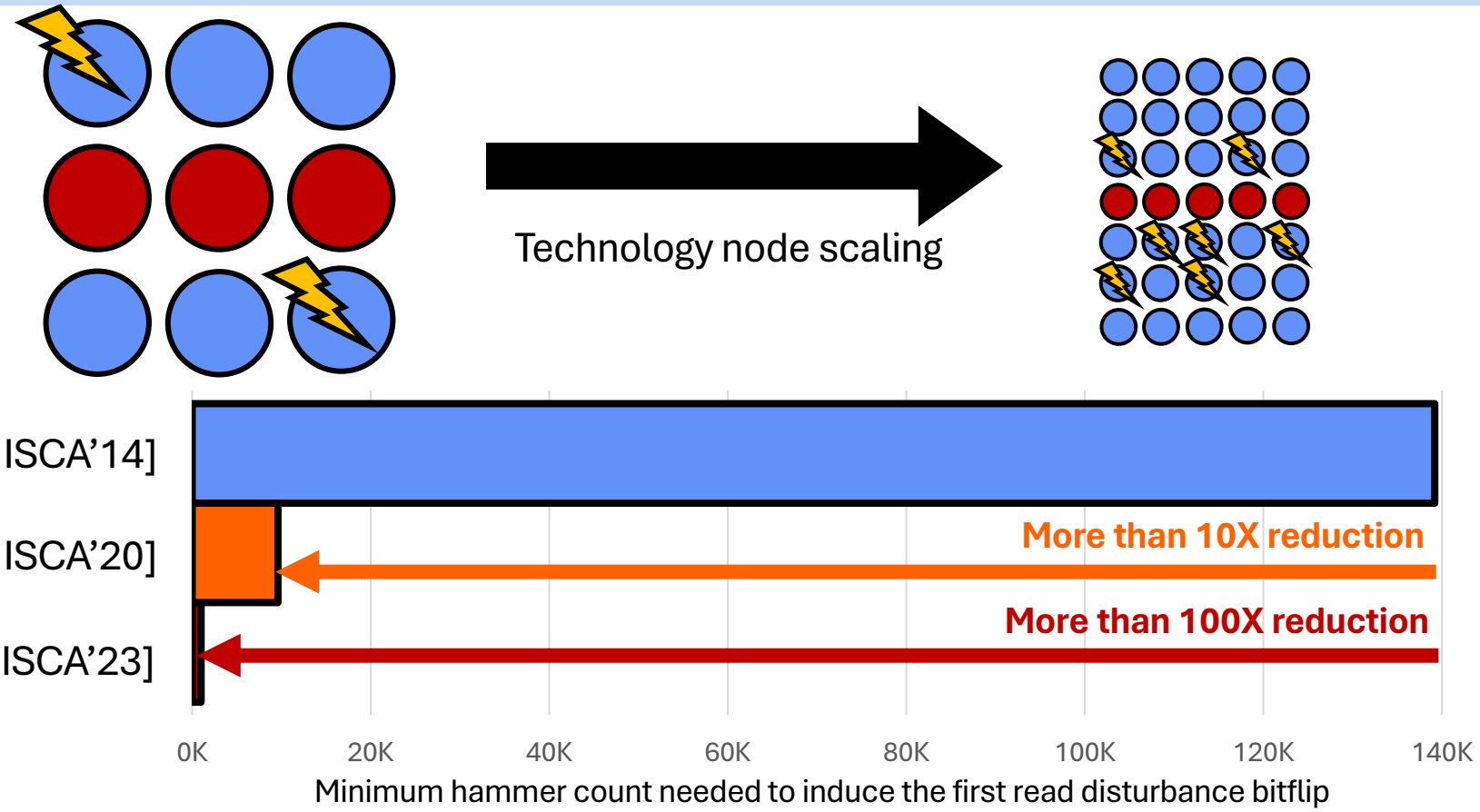
Mitigating Read Disturbance (II): Preventive Refresh



Defense techniques **track row activation counts** (of aggressor rows) to **preventively refresh** potential victim rows

[Bostanci+ HPCA'24]

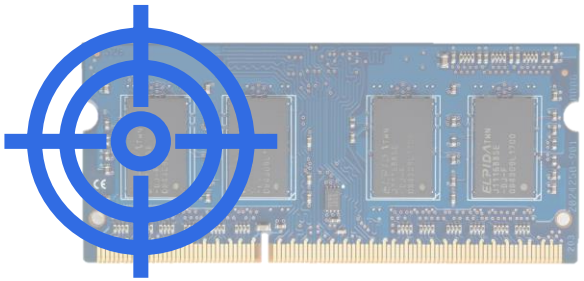
Read Disturbance is Getting Worse



DRAM cells become **increasingly more vulnerable** to **read disturbance,** requiring **low-cost defenses**

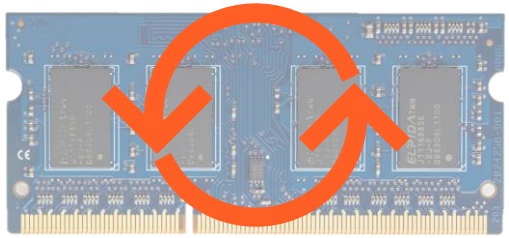
Latest Industry Defenses for RowHammer

Recent DDR5 specifications introduce two new defense frameworks to mitigate RowHammer bitflips:



Per-row activation counters
to accurately track aggressor rows' activation counts

Per Row Activation Counting (PRAC)
as of April 2024



A command to
provide DRAM with the time
for performing preventive actions

Refresh Management (RFM)
before 2024

Problem

Do industrial and academic RowHammer defenses introduce **new covert and side channel vulnerabilities?**

Unfortunately, yes.

Talk Outline

I. Motivation and Problem

II. Key Observation

III. LeakyHammer and Case Studies

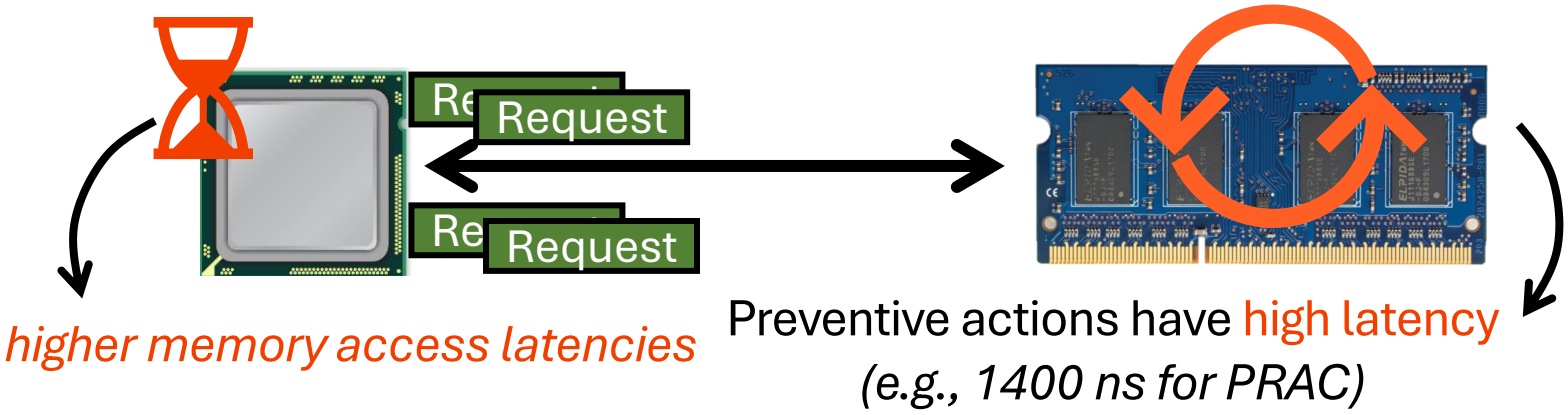
IV. Mitigating LeakyHammer

V. Conclusion

Key Observation (I)

- Preventive actions have **two fundamental features** that enable covert channels and side channels:

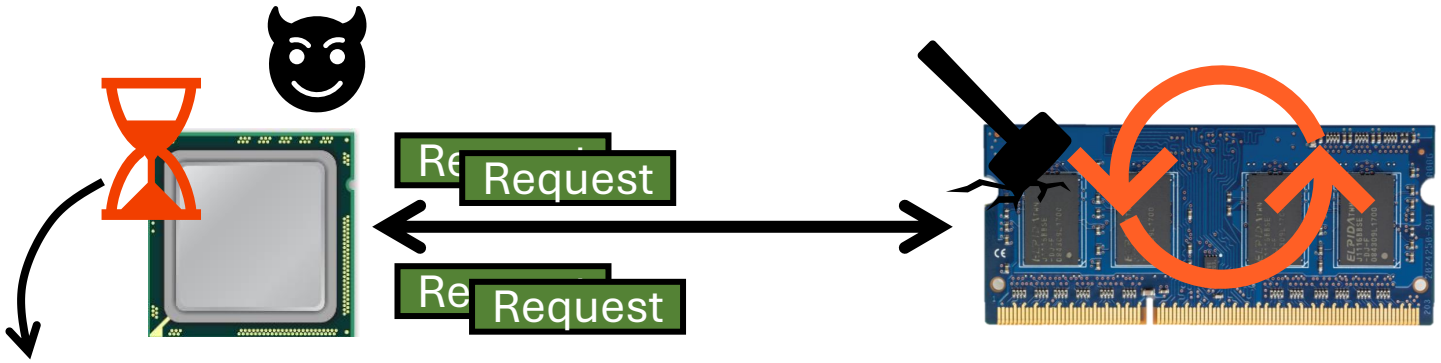
1 Preventive actions often result in **high memory access latencies** for regular memory requests



Key Observation (II)

- Preventive actions have **two fundamental features** that enable covert channels and side channels:

2 A user can leverage memory access patterns to **intentionally trigger a preventive action**



higher memory access latencies

Talk Outline

I. Motivation and Problem

II. Key Observation

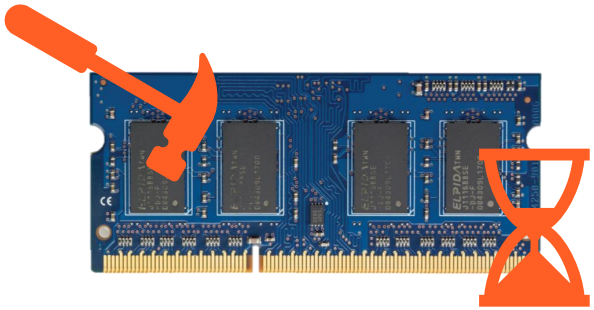
III. LeakyHammer and Case Studies

IV. Mitigating LeakyHammer

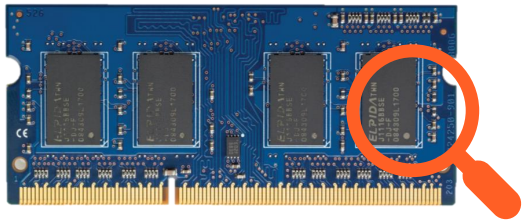
V. Conclusion

LeakyHammer

A new class of attacks that exploit
RowHammer defense-induced memory latency differences



Deterministically and intentionally
impose observable latency overheads
on requests via preventive actions



Accurately infer victims'
memory access patterns
that result in preventive actions

LeakyHammer Case Studies

CS1: PRAC-Based Covert Channel Attack

CS2: RFM-Based Covert Channel Attack

CS3: PRAC-Based Website Fingerprinting

Evaluation Methodology

- **Environment:** System simulation using gem5 [Lowe-Power+ '20] integrated with cycle-level DRAM simulator Ramulator 2.0 [Kim+ CAL'15, Luo+ CAL'24]

- **System Configuration:**

Processor	1-, 2-, and 4-core, out-of-order, 3GHz clock frequency
DRAM	DDR5, 1 channel, 2 rank/channel, 8 bank groups, 4 banks/bank group, 128K rows/bank
Memory Ctrl.	64-entry read and write requests queues, Scheduling policy: FR-FCFS with a column cap of 16 Last-level cache 4 MiB (per-core), 16-way
PRAC	N_{BO}=128 activations, 4 RFMs per back-off
PRFM	T_{RFM}=40 activations

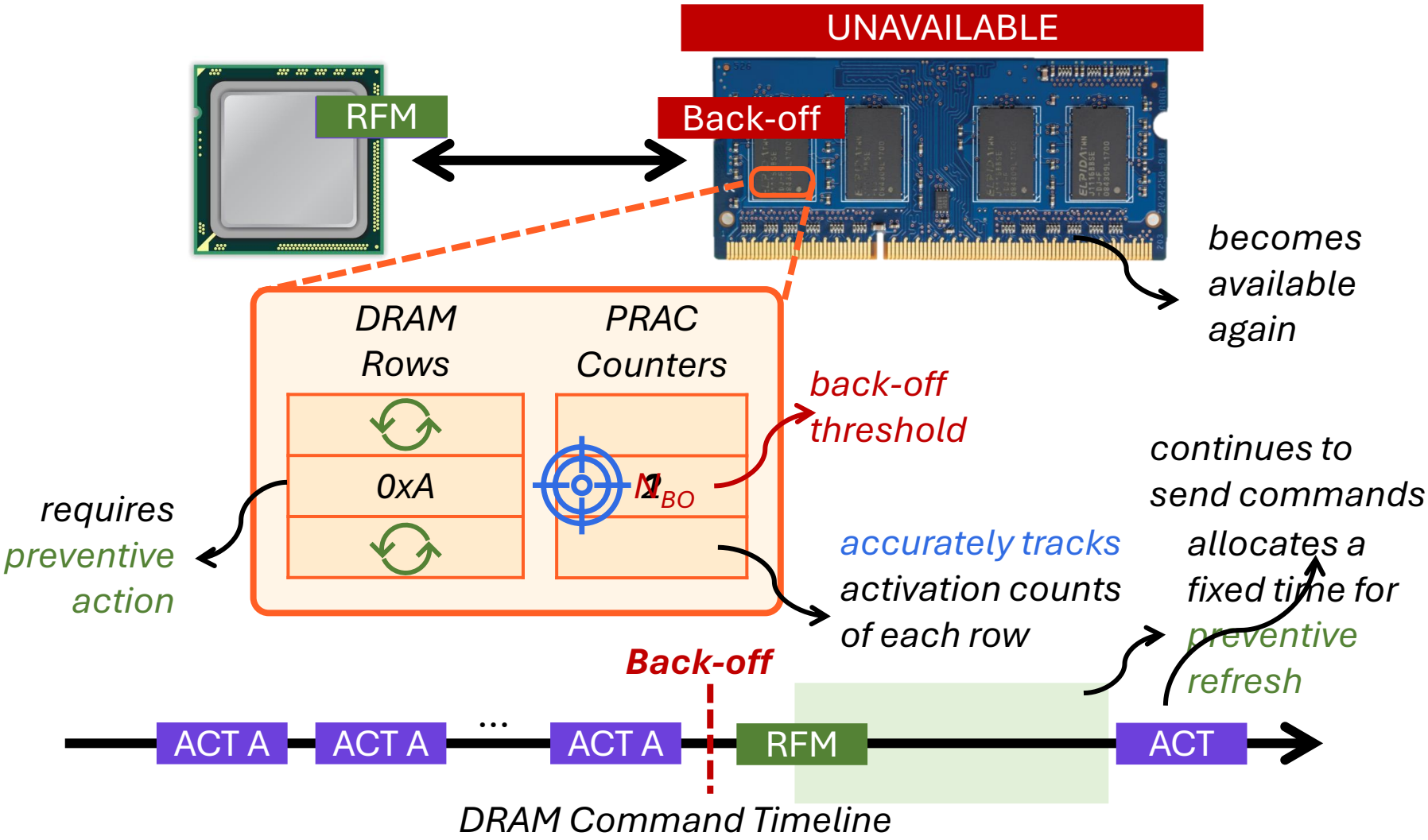
LeakyHammer Case Studies

CS1: PRAC-Based Covert Channel Attack

CS2: RFM-Based Covert Channel Attack

CS3: PRAC-Based Website Fingerprinting

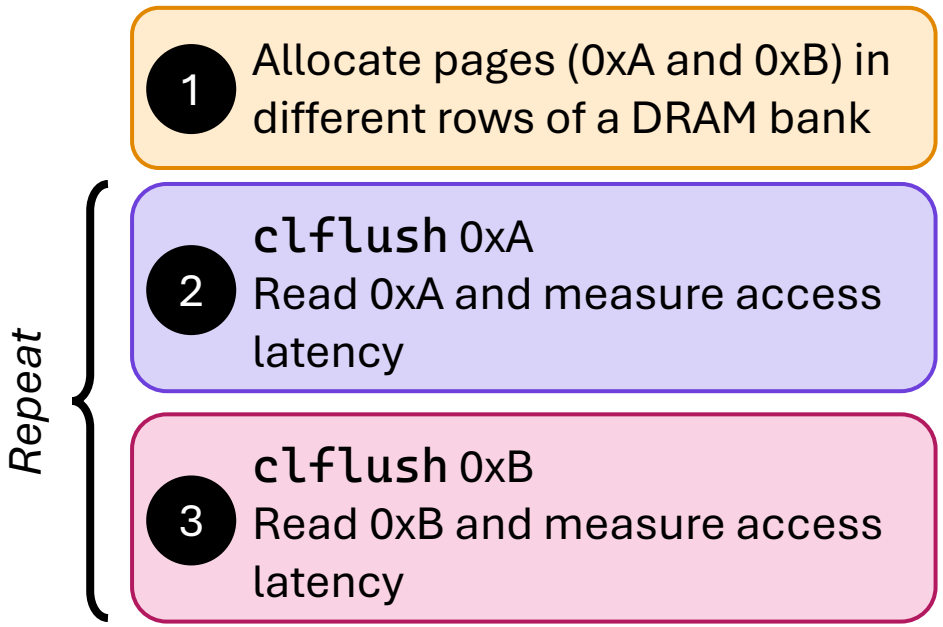
Per Row Activation Counting in DDR5

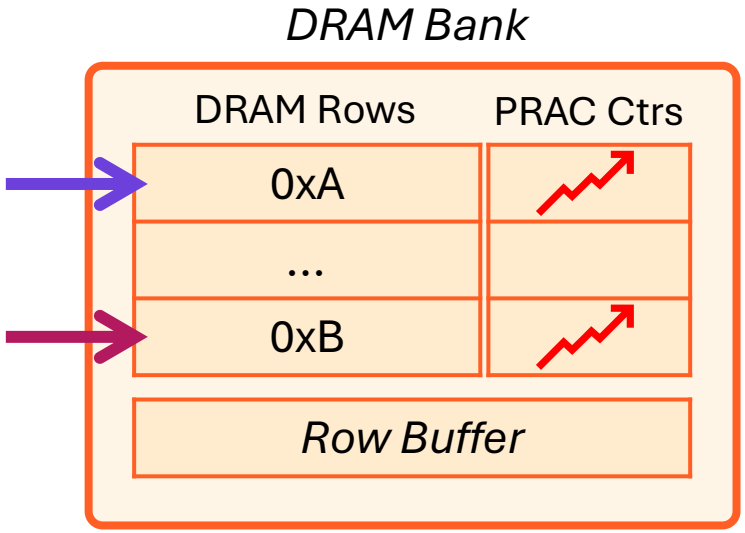


Observing PRAC-Induced Latencies (I)

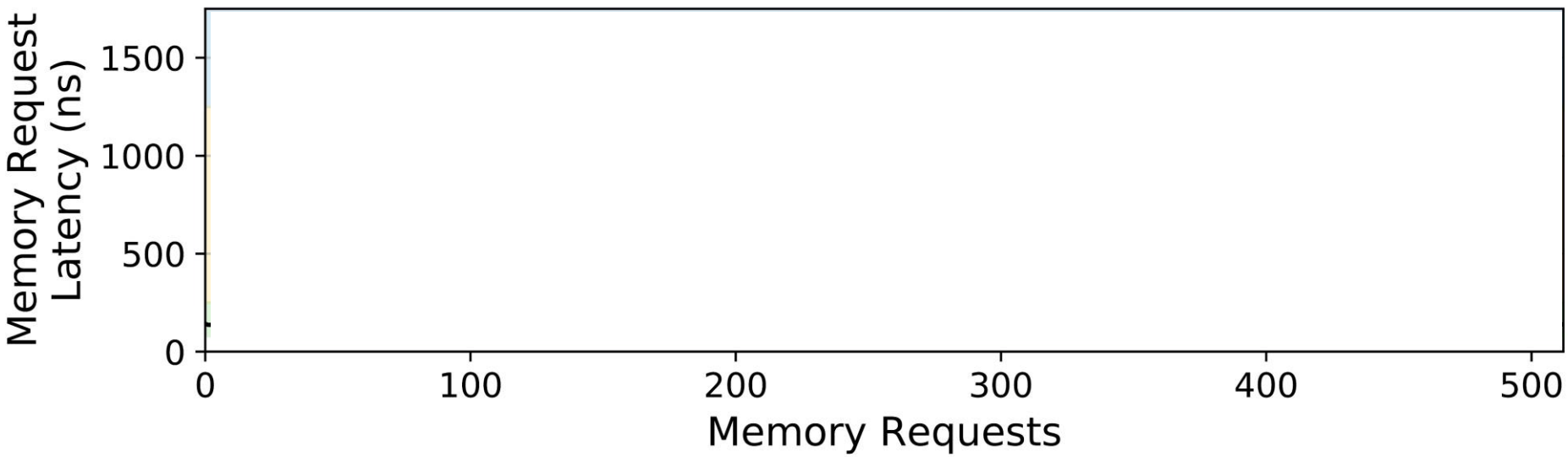
We test if an application can:

- intentionally trigger a PRAC back-off
- observe PRAC-induced higher memory access latencies

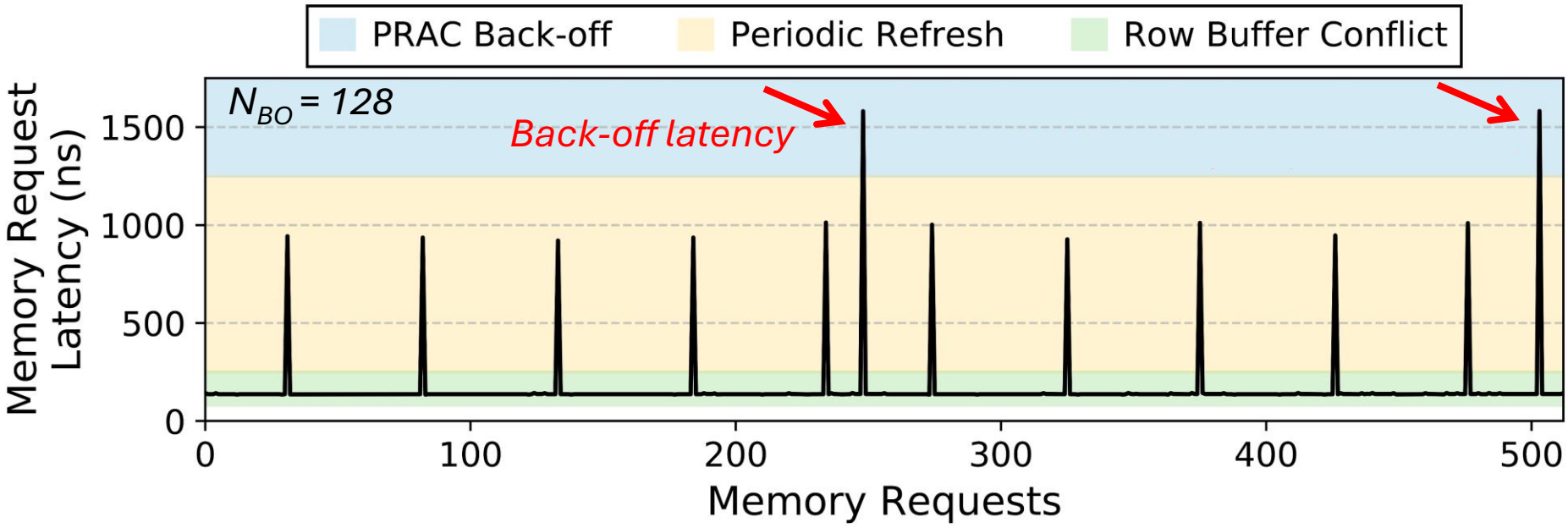




Observing PRAC-Induced Latencies (II)



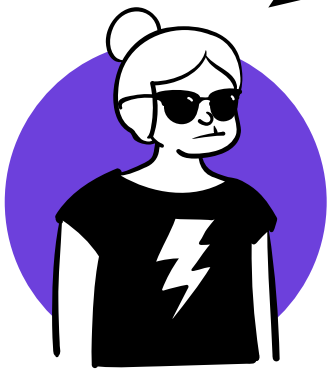
Observing PRAC-Induced Latencies (II)



The observed back-off latency is consistent with the specification and can be distinguished from regular memory operations

PRAC-Based Covert Channel: Attack Overview

10101001



Sender

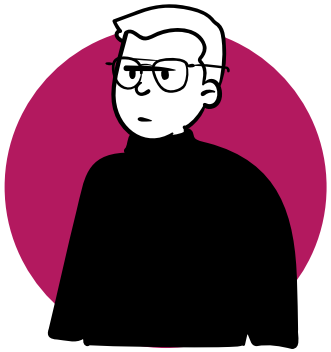
Encoding

logic-1: back-off

logic-0: no-back-off

DRAM Bank

DRAM Rows	*Ctrs*
R_{Sender}	
...	
$R_{Receiver}$	
Row Buffer	



Receiver

Only 1 bit will be transmitted

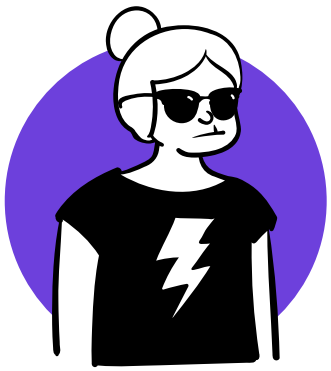


PRAC-Based Covert Channel: Attack Overview

Encoding

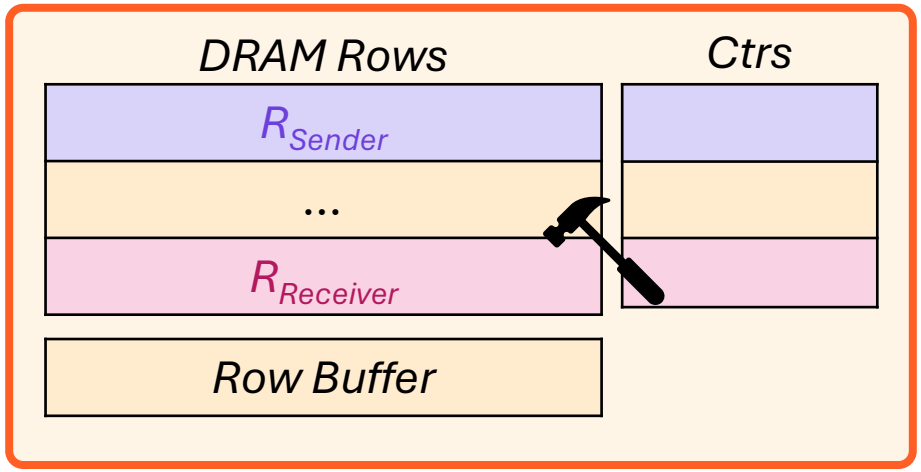
logic-1: back-off

logic-0: no-back-off



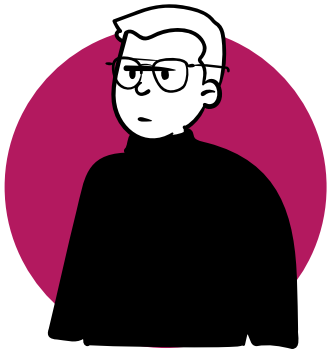
Sender

DRAM Bank



Loop:

time(access $R_{Receiver}$)



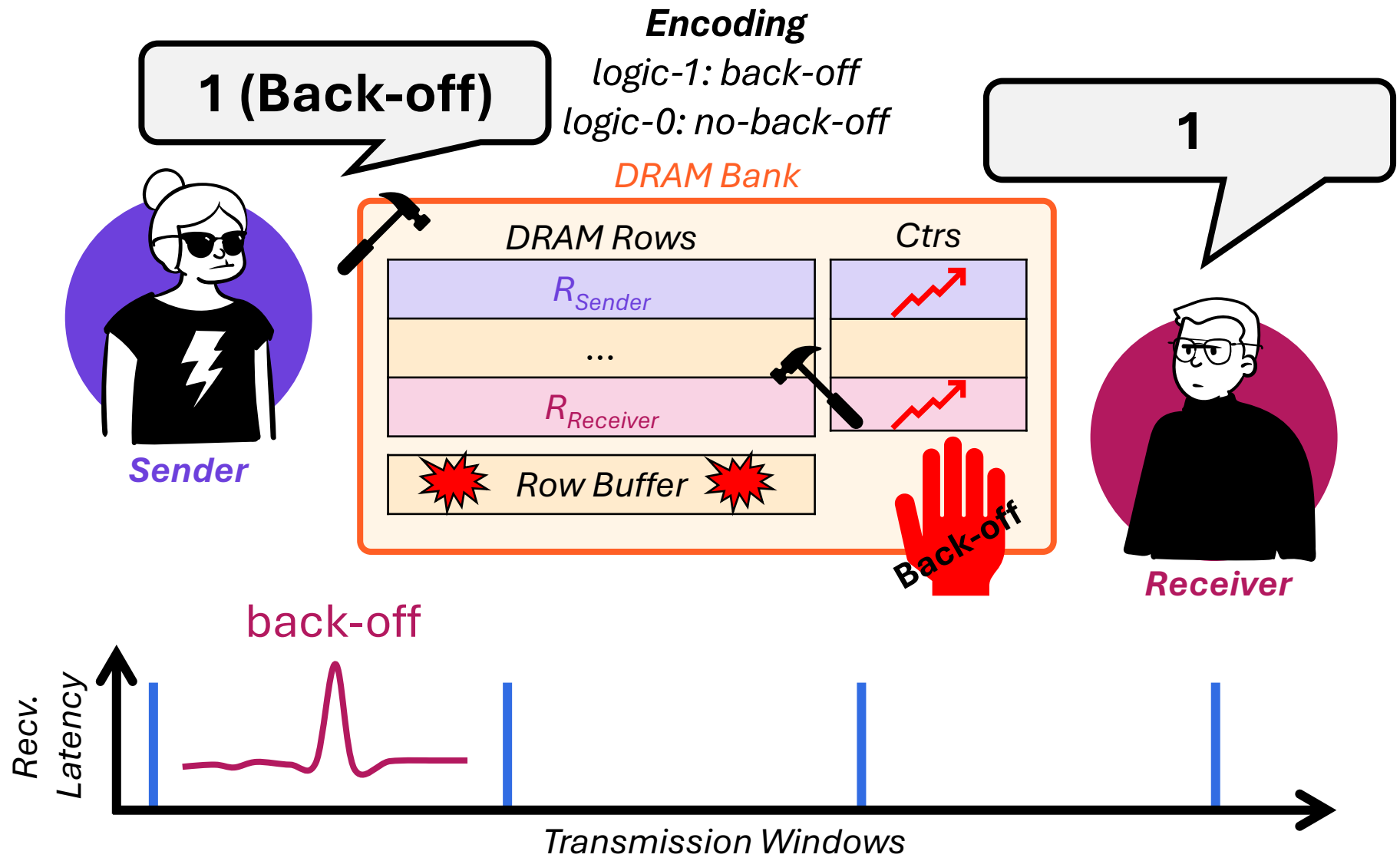
Receiver

Only 1 bit will be transmitted

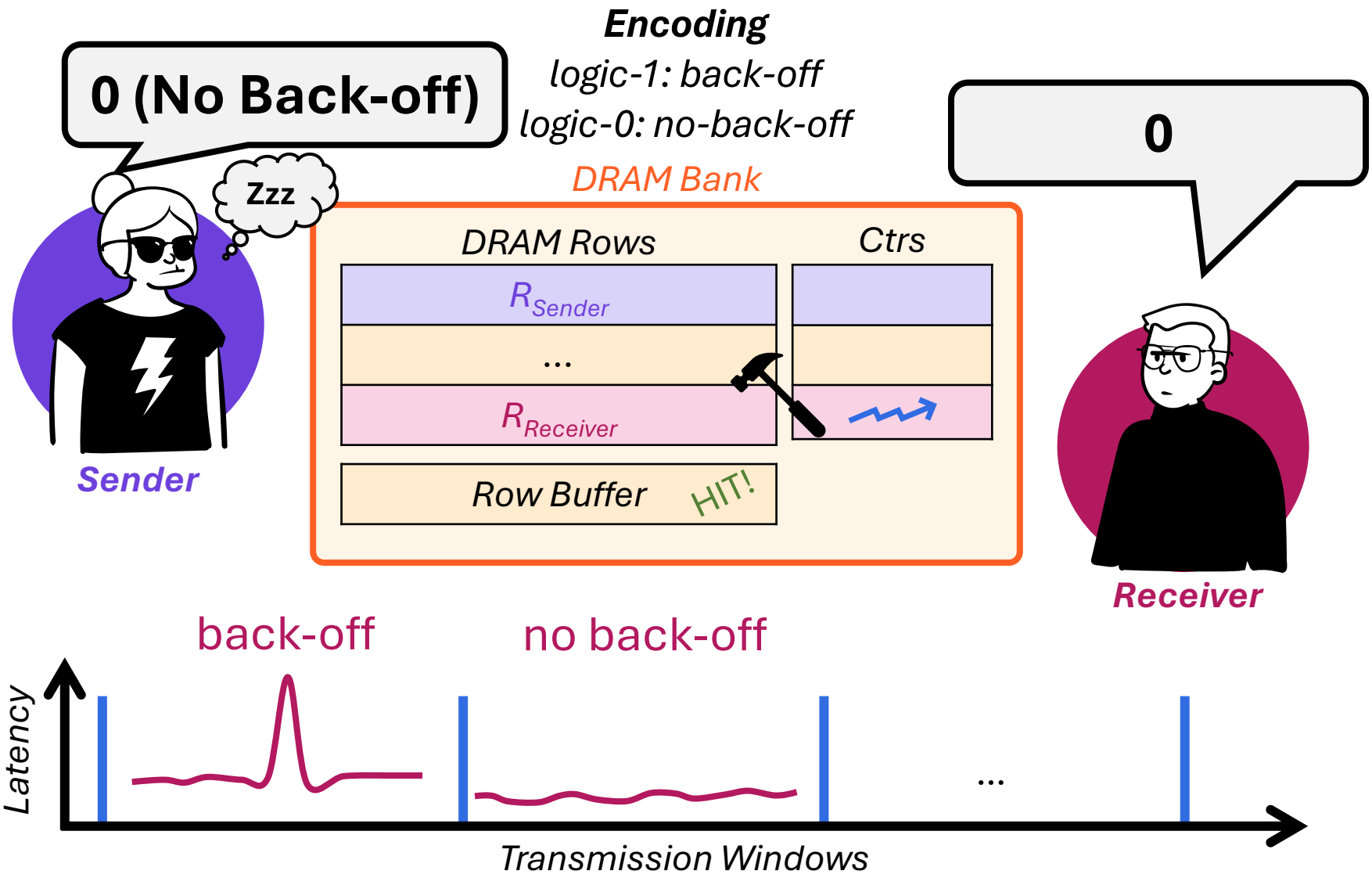
used to determine the bit value



PRAC-Based Covert Channel: Attack Overview

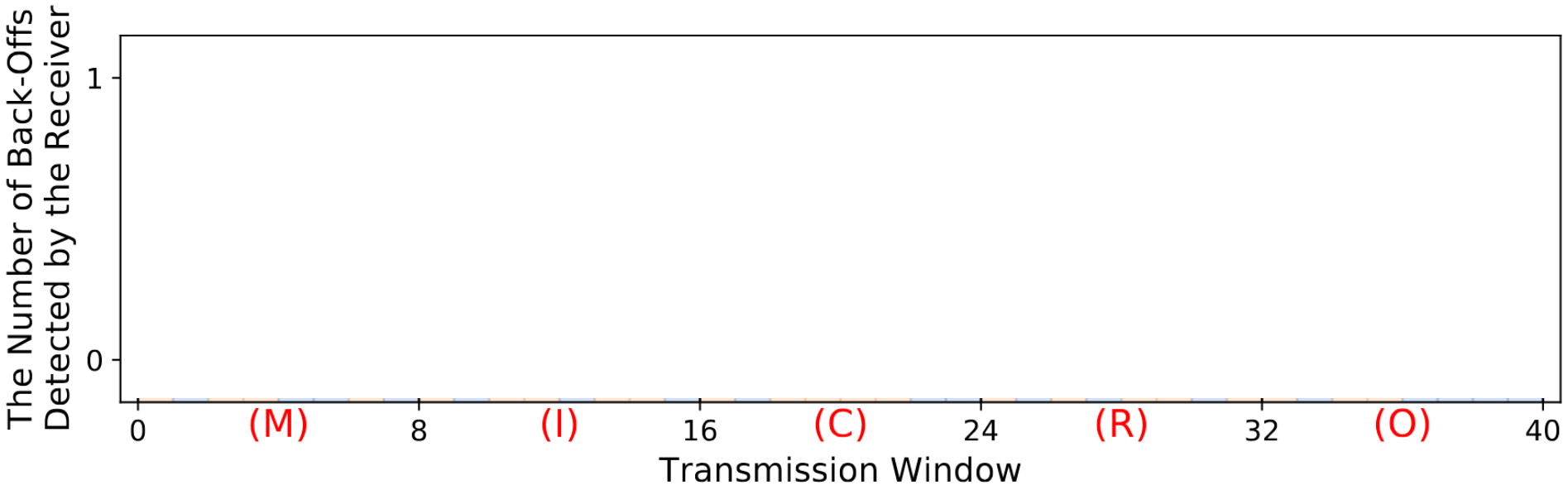


PRAC-Based Covert Channel: Attack Overview



PRAC-Based Covert Channel: Proof-of-Concept and Channel Capacity

- Validation with transmitting a 40-bit message

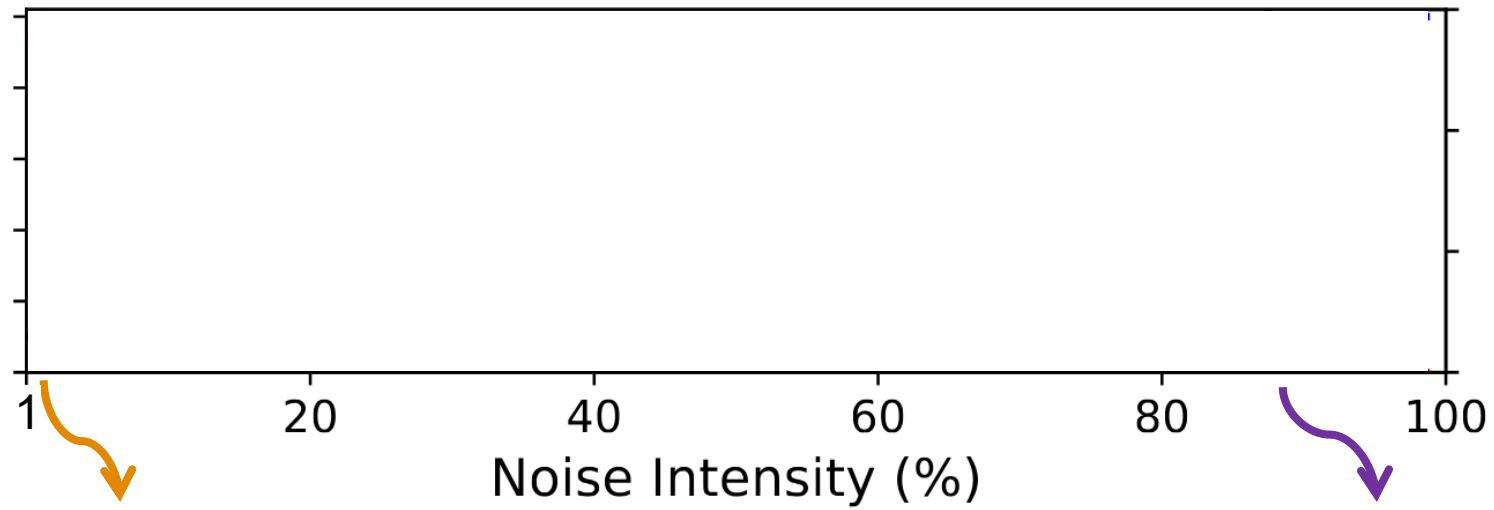


The receiver **successfully decodes** the sender's message

The attack achieves **39.0 Kbps raw bit rate**

PRAC-Based Covert Channel: Noise Analysis

- We use a microbenchmark that causes back-offs with varying frequencies to simulate system noise

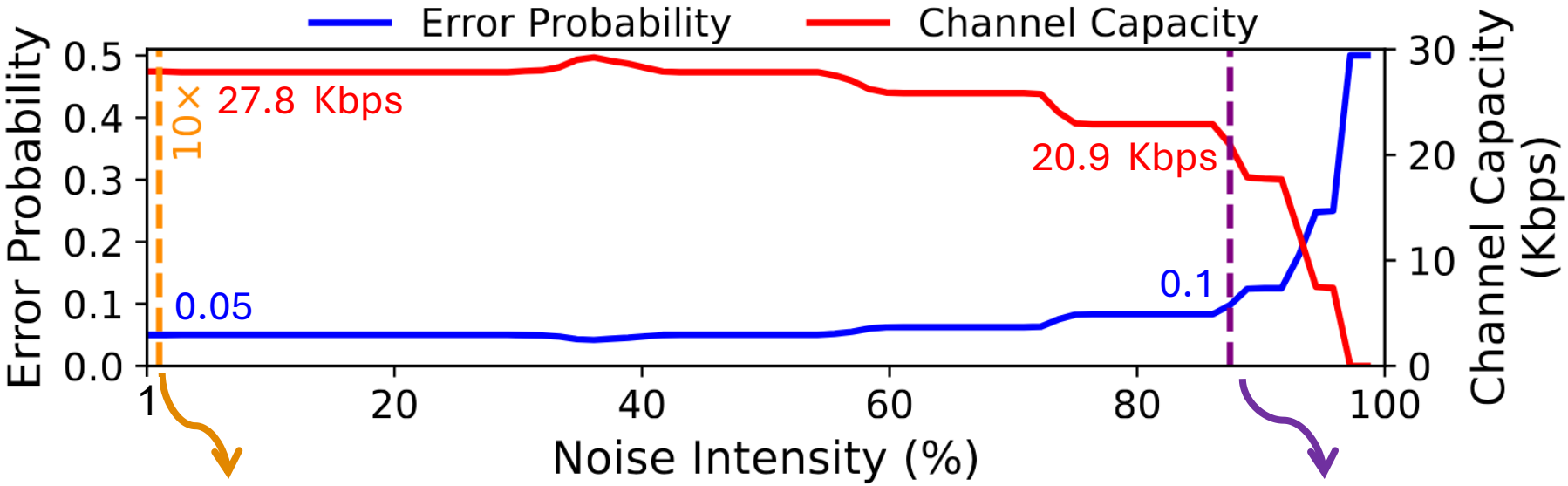


1% = *10 x 4-core memory-intensive workload*

Similar back-off intensity to memory performance attacks
[Moscibroda+, USENIX Security'07]

PRAC-Based Covert Channel: Noise Analysis

- We use a microbenchmark that causes back-offs with varying frequencies to simulate system noise



1% = *10 x 4-core memory-intensive workload*

Similar back-off intensity to memory performance attacks

[Moscibroda+, USENIX Security'07]

The PRAC-based covert channel attack **maintains its capacity** until very high noise intensity values

LeakyHammer Case Studies

CS1: PRAC-Based Covert Channel Attack

CS2: RFM-Based Covert Channel Attack

CS3: PRAC-Based Website Fingerprinting

RFM-Based Covert Channel Summary

We exploit an RFM-based RowHammer defense that tracks **the number of row activations per DRAM bank** as described in the DDR5 standard

We observe that applications **can intentionally trigger an RFM command** and can detect it by **observing the high memory latency**

The attack successfully transmits the message

Due to per DRAM bank activation counting, applications can trigger RFM more easily, making the channel **more susceptible to noise**

RFM-Based Covert Channel Summary



Understanding and Mitigating Covert Channel and Side Channel Vulnerabilities Introduced by RowHammer Defenses

F. Nisa Bostancı[1] Oğuzhan Canpolat[1,2] Ataberk Olgun[1] İsmail Emir Yüksel[1]
Konstantinos Kanellopoulos[1] Mohammad Sadrosadati[1] A. Giray Yağlıkçı[1,3] Onur Mutlu[1]
[1]ETH Zürich [2]TOBB ETÜ [3]CISPA

Oct 2025

DRAM chips are increasingly vulnerable to read disturbance phenomena (e.g., RowHammer and RowPress), where repeatedly accessing or keeping open a DRAM row causes bitflips in nearby rows, due to DRAM density scaling. Attackers can exploit RowHammer bitflips in real systems to compromise security, which has motivated many prior works on RowHammer defenses. To enable such defenses, recent DDR specifications introduce new defense frameworks (e.g., PRAC and RFM). For robust (i.e., secure, safe, and reliable) operation, it is critical to analyze security implications of widely-adopted RowHammer

rows) [1–14]. RowHammer [1,2,4,14] is a type of read disturbance phenomenon, where a victim row can experience bitflips when at least one nearby row is hammered more times than a threshold, called the RowHammer threshold (N_{RH}). Modern DRAM chips become more vulnerable to RowHammer as DRAM technology node size becomes smaller [1,2,4,14–19]. *RowPress* [13] is another example DRAM read disturbance phenomenon that amplifies bitflips by keeping the aggressor row open for longer, thereby causing more disturbance with each activation. Prior works show that attackers can leverage

https://arxiv.org/abs/2503.17891

LeakyHammer Case Studies

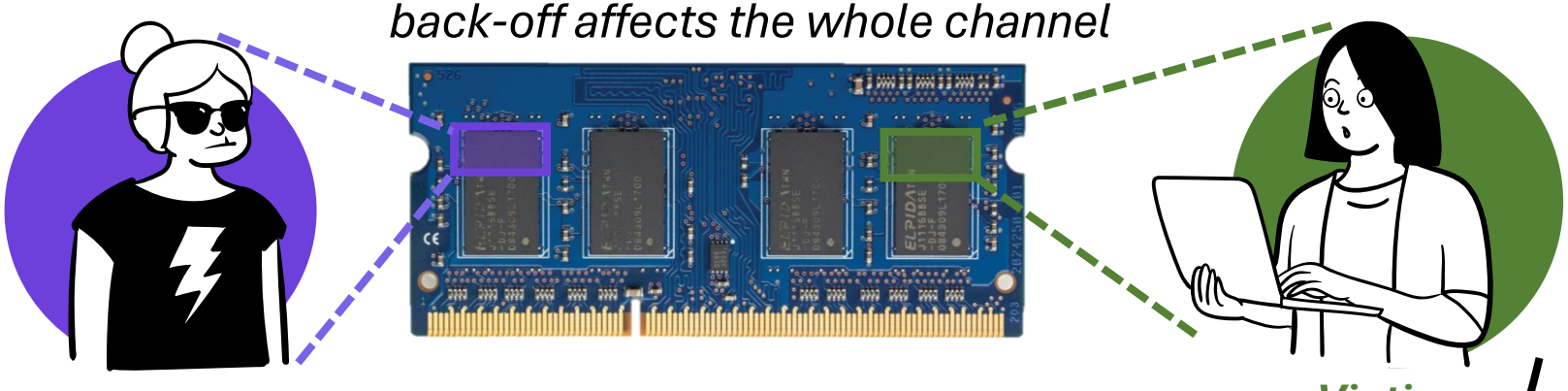
CS1: PRAC-Based Covert Channel Attack

CS2: RFM-Based Covert Channel Attack

CS3: PRAC-Based Website Fingerprinting

PRAC-Based Website Fingerprinting Attack

No need to colocate in the same bank as back-off affects the whole channel



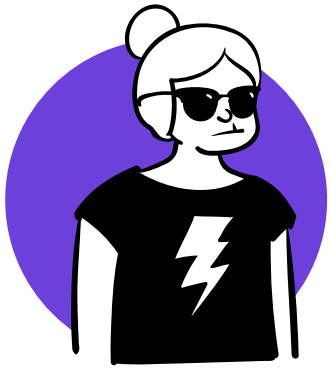
Attacker

Determines which website the victim loads by only observing own memory request latencies

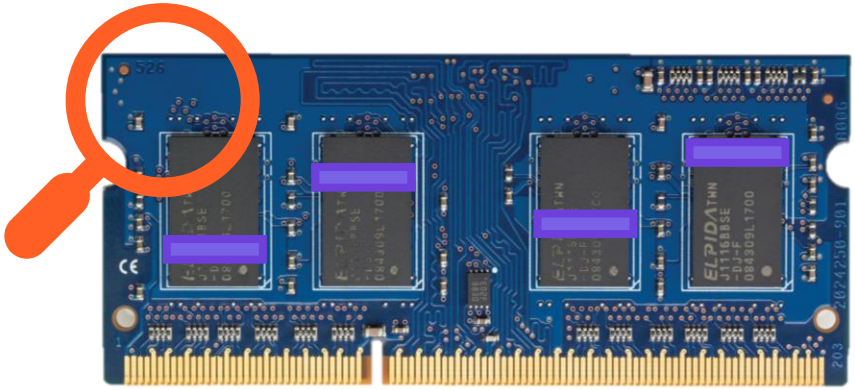
Victim

Loads websites using a web browser

How to Fingerprint

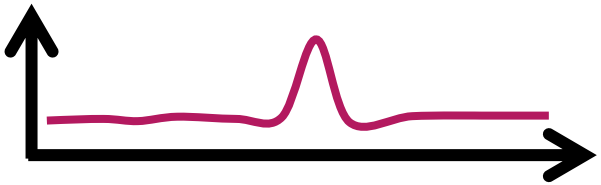


Attacker



1

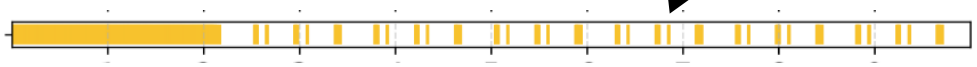
allocate multiple test rows to avoid triggering back-off while observing



2

access test rows continuously and measure access latencies to detect back-offs

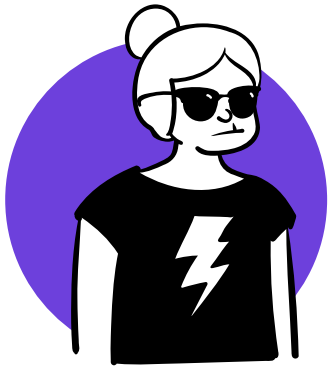
each line is a back-off



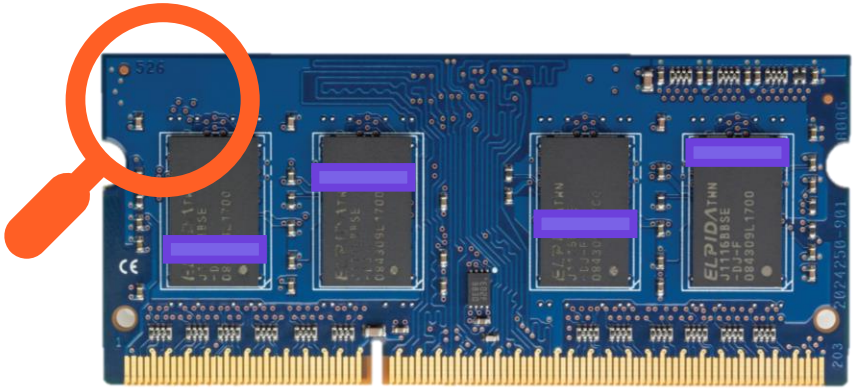
3

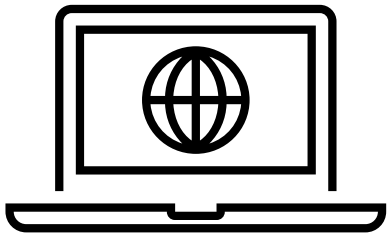
create fingerprint by only using the timestamps of back-offs

Preparing for the Attack: Creating a Database

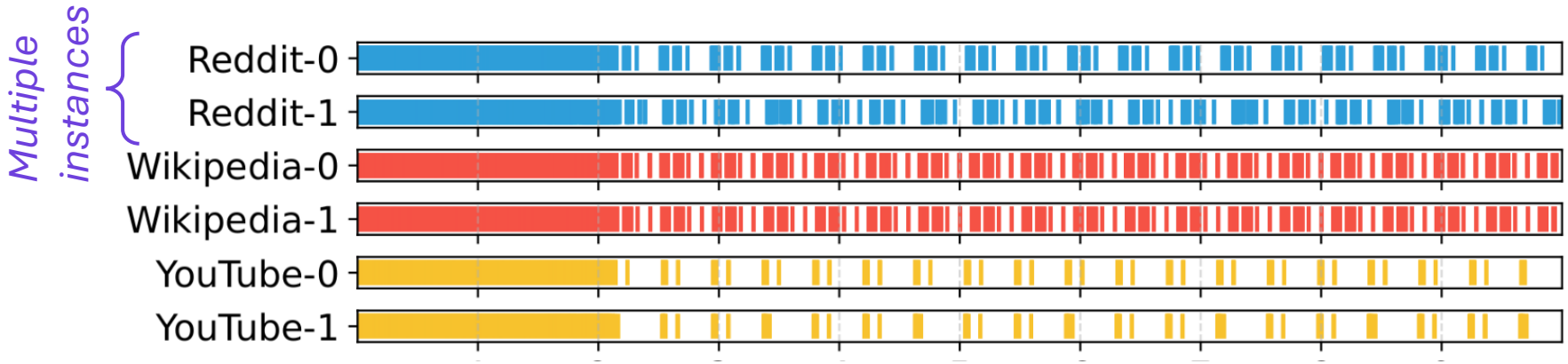


Attacker

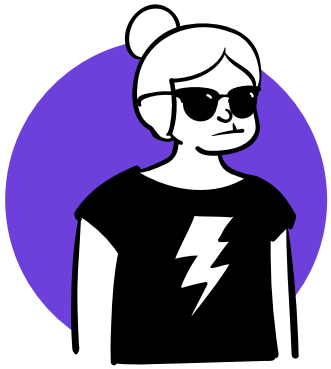




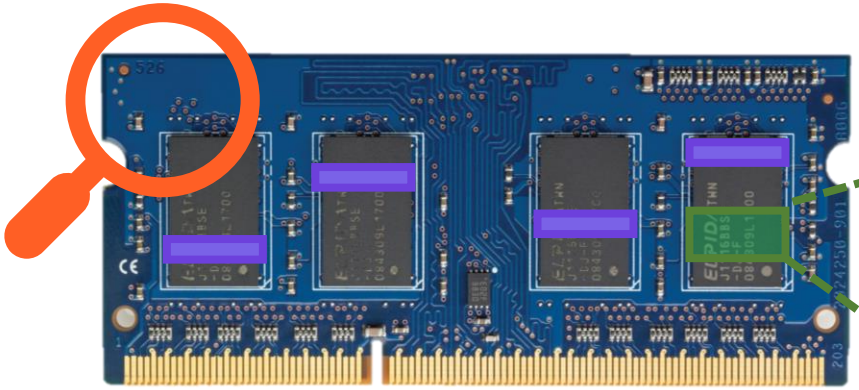
fingerprint many websites
and create a fingerprint database



Launching the Attack



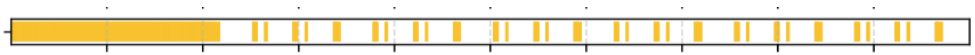
Attacker

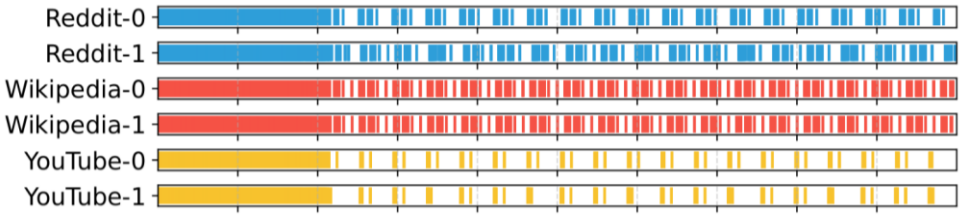




Victim

1 Get *a fingerprint* when *the victim loads a website*





2 *compare with the database and determine the website*

↓

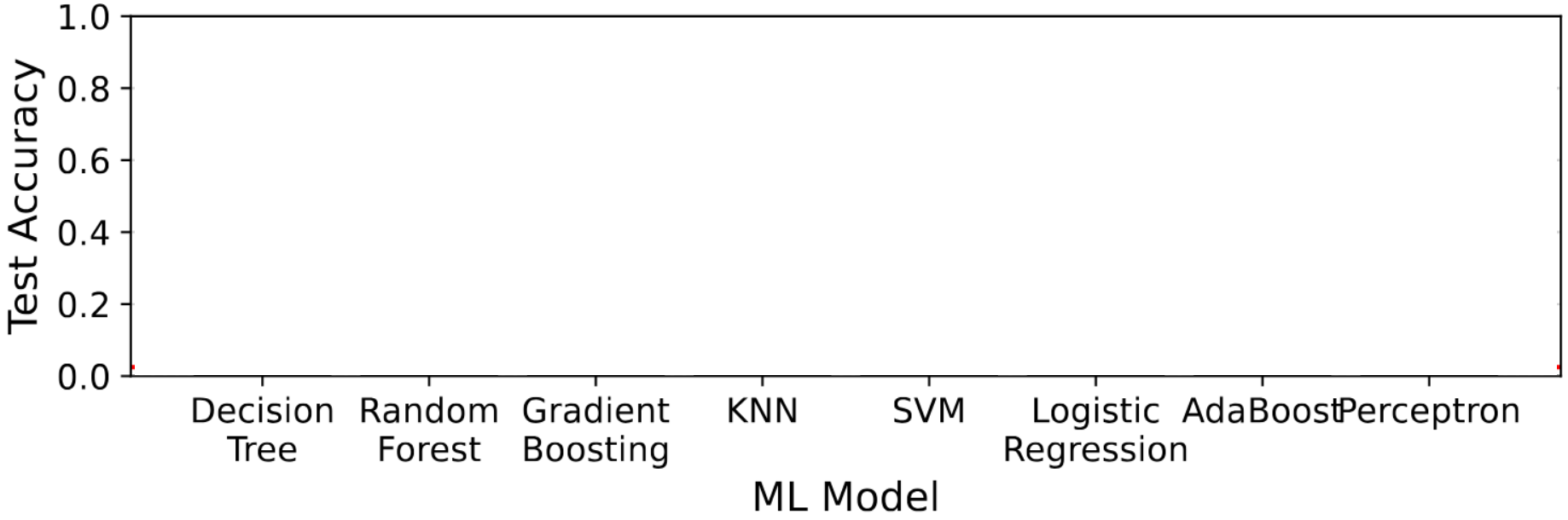
`Website: YouTube`

PRAC-Based Website Fingerprinting: Evaluation

- We collect back-off fingerprints of a website by loading it with a browser and getting a back-off trace with the attacker routine
  - 40 most-visited websites from Semrush and Similarweb top sites

- A machine learning approach to classify websites:
  - Simple and lightweight machine learning models (e.g., decision tree, random forest, gradient boosting, k-nearest neighbors)

PRAC-Based Website Fingerprinting: Test Accuracy



Using back-off-based fingerprints, **classical machine learning models** can **classify websites with up to 75% accuracy**

Talk Outline

I. Motivation and Problem

II. Key Observation

III. LeakyHammer and Case Studies

IV. Mitigating LeakyHammer

V. Conclusion

Mitigating LeakyHammer

- We propose three countermeasures against LeakyHammer:

Fixed-Rate RFM (FR-RFM)

eliminates LeakyHammer by decoupling preventive actions from memory access patterns of running applications

Randomly Initialized Activation Counters (RIAC)

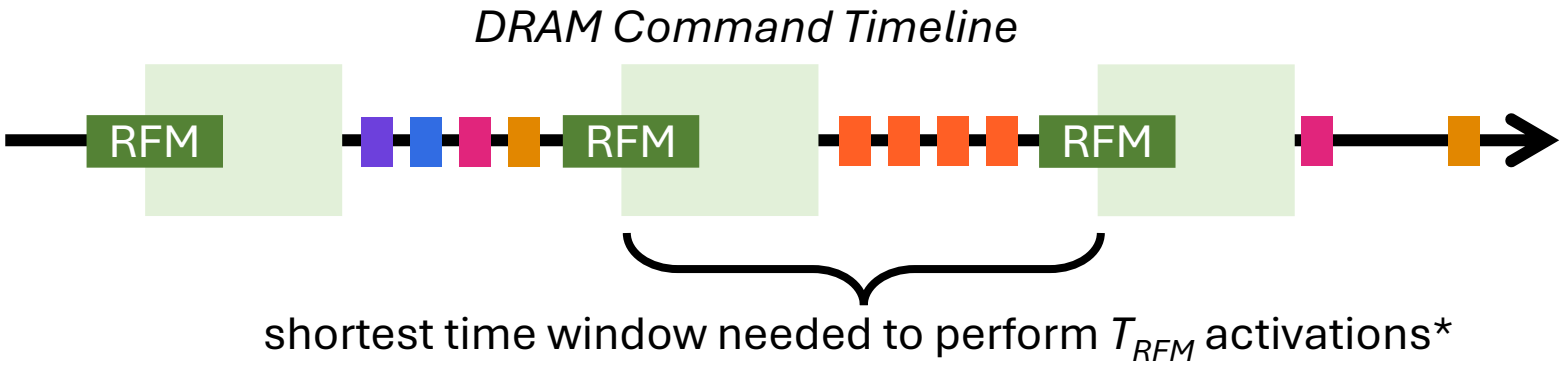
reduces channel capacity with a randomization method

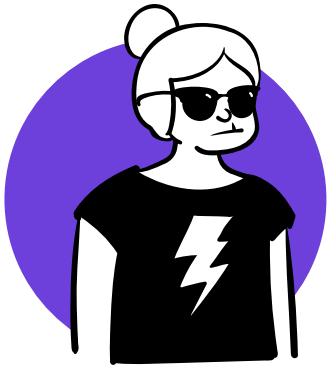
Bank-Level PRAC (PRAC-Bank)

performs bank-level preventive actions to limit the attack scope to that of existing DRAM-based attacks

Fixed-Rate RFM (FR-RFM)

- Periodically performs **a preventive action**,
  - **independently** from **the memory access patterns**





- **A sender** *cannot* cause preventive actions intentionally to send messages
- **An attacker** *cannot* observe a victim's memory access patterns that lead to a preventive action

* Security analysis available in the paper

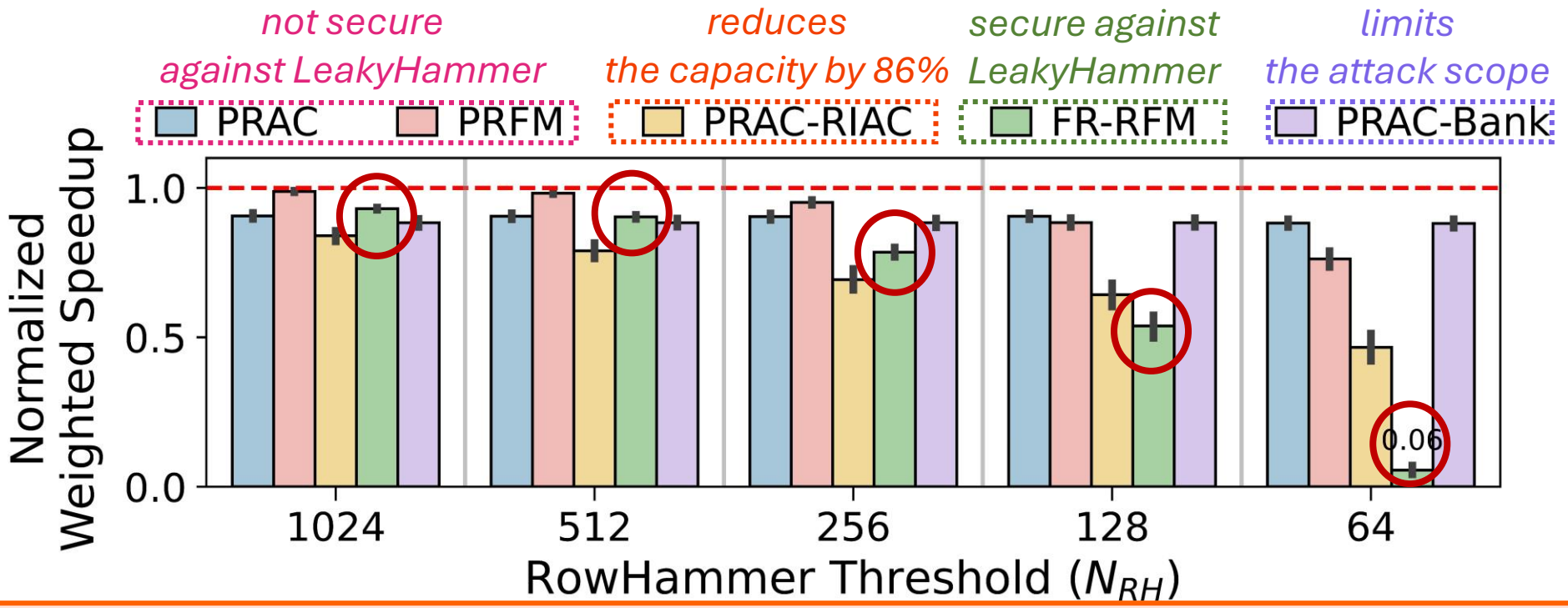
Evaluating LeakyHammer Countermeasures

- 60 four-core workloads: SPEC2017 and SPEC2006



Evaluating LeakyHammer Countermeasures

- 60 four-core workloads: SPEC2017 and SPEC2006



FR-RFM mitigates LeakyHammer and performs similar to PRAC and PRFM for near-future N_{RH} values (e.g., 1024)

As N_{RH} decreases, other countermeasures have lower performance overheads

More in the Paper

- More details of LeakyHammer
  - Operational details on case studies (e.g., RFM-based covert channel)
  - Multibit LeakyHammer covert channels
  - LeakyHammer countermeasures

- More results and analyses
  - LeakyHammer with other RowHammer mitigations
  - Impact of real workloads on LeakyHammer
  - Sensitivity of PRAC-based covert channel
    - Impact of RFM commands per back-off, RowHammer defense latency, larger cache hierarchy and data prefetchers
  - Comparison against existing cache and memory-based timing channels

More in the Paper



Understanding and Mitigating Covert Channel and Side Channel Vulnerabilities Introduced by RowHammer Defenses

F. Nisa Bostancı[1] Oğuzhan Canpolat[1,2] Ataberk Olgun[1] İsmail Emir Yüksel[1]
Konstantinos Kanellopoulos[1] Mohammad Sadrosadati[1] A. Giray Yağlıkçı[1,3] Onur Mutlu[1]
[1]ETH Zürich [2]TOBB ETÜ [3]CISPA

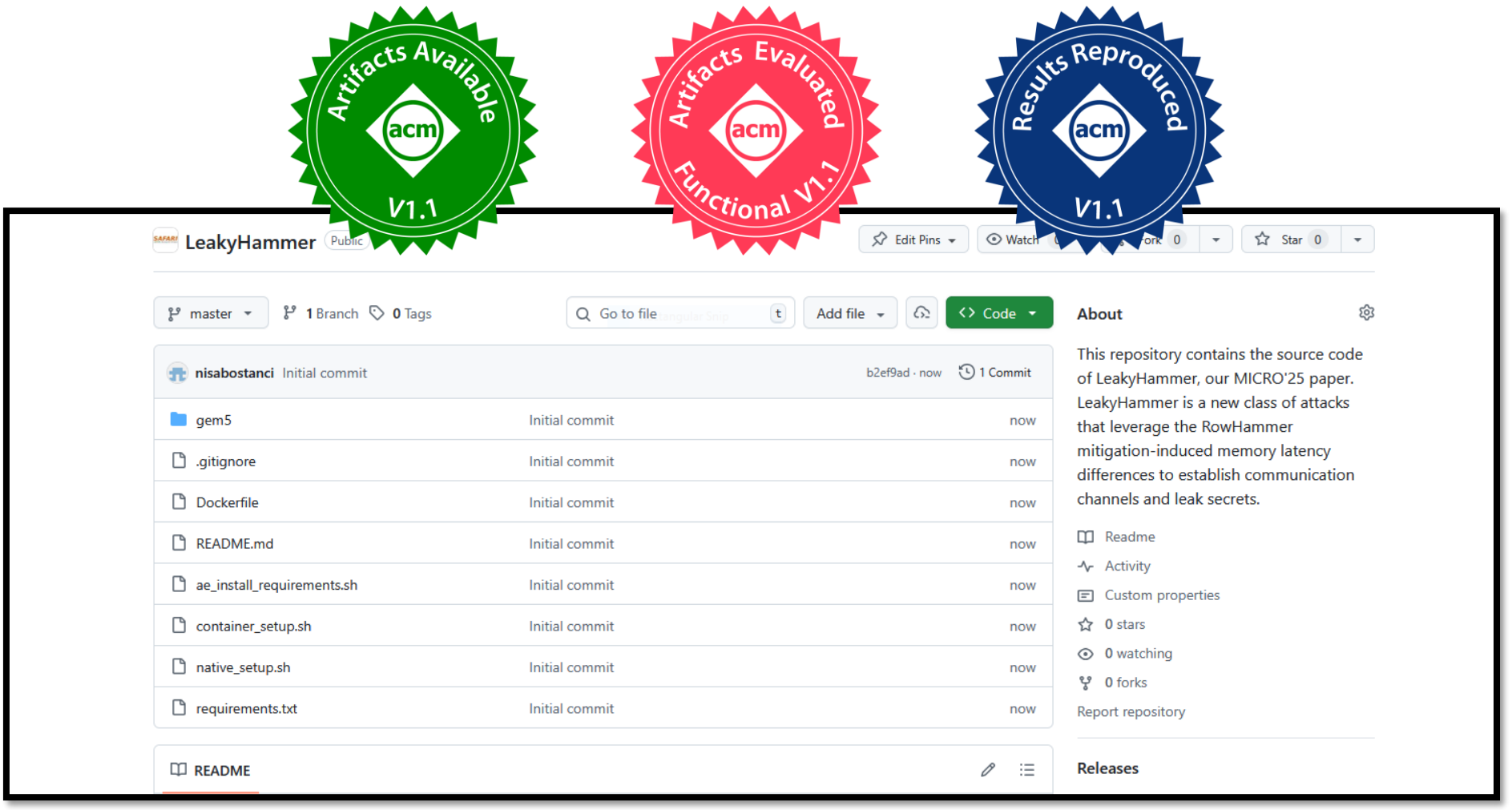
Oct 2025

DRAM chips are increasingly vulnerable to read disturbance phenomena (e.g., RowHammer and RowPress), where repeatedly accessing or keeping open a DRAM row causes bitflips in nearby rows, due to DRAM density scaling. Attackers can exploit RowHammer bitflips in real systems to compromise security, which has motivated many prior works on RowHammer defenses. To enable such defenses, recent DDR specifications introduce new defense frameworks (e.g., PRAC and RFM). For robust (i.e., secure, safe, and reliable) operation, it is critical to analyze security implications of widely-adopted RowHammer

rows) [1–14]. RowHammer [1,2,4,14] is a type of read disturbance phenomenon, where a victim row can experience bitflips when at least one nearby row is hammered more times than a threshold, called the RowHammer threshold (N_{RH}). Modern DRAM chips become more vulnerable to RowHammer as DRAM technology node size becomes smaller [1,2,4,14–19]. *RowPress* [13] is another example DRAM read disturbance phenomenon that amplifies bitflips by keeping the aggressor row open for longer, thereby causing more disturbance with each activation. Prior works show that attackers can leverage

https://arxiv.org/abs/2503.17891

LeakyHammer is Open Source and Artifact Evaluated



https://github.com/CMU-SAFARI/LeakyHammer

Talk Outline

I. Motivation and Problem

II. Key Observation

III. LeakyHammer and Case Studies

IV. Mitigating LeakyHammer

V. Conclusion

Conclusion

We introduce **LeakyHammer**, a new class of timing attacks that exploit RowHammer defense-induced memory latency differences

We systematically analyze two latest industry defenses to RowHammer
- Build two covert communication channels
- Demonstrate a website fingerprinting attack that identifies visited websites with high accuracy

We propose and evaluate three countermeasures
- Completely mitigating LeakyHammer with low performance overhead in near-future DRAM-based systems
- In systems that are highly RowHammer-vulnerable, countermeasures that reduce channel capacity have lower performance overhead

https://github.com/CMU-SAFARI/LeakyHammer

arXiv GitHub







Understanding and Mitigating Covert Channel and Side Channel Vulnerabilities Introduced by RowHammer Defenses

F. Nisa Bostancı

O. Canpolat A. Olgun I. E. Yüksel K. Kanellopoulos
M. Sadrosadati A. G. Yağlıkçı O. Mutlu

ETH zürich SAFARI CISPA HELMHOLTZ CENTER FOR INFORMATION SECURITY

arXiv GitHub







Understanding and Mitigating Covert Channel and Side Channel Vulnerabilities Introduced by RowHammer Defenses

F. Nisa Bostancı

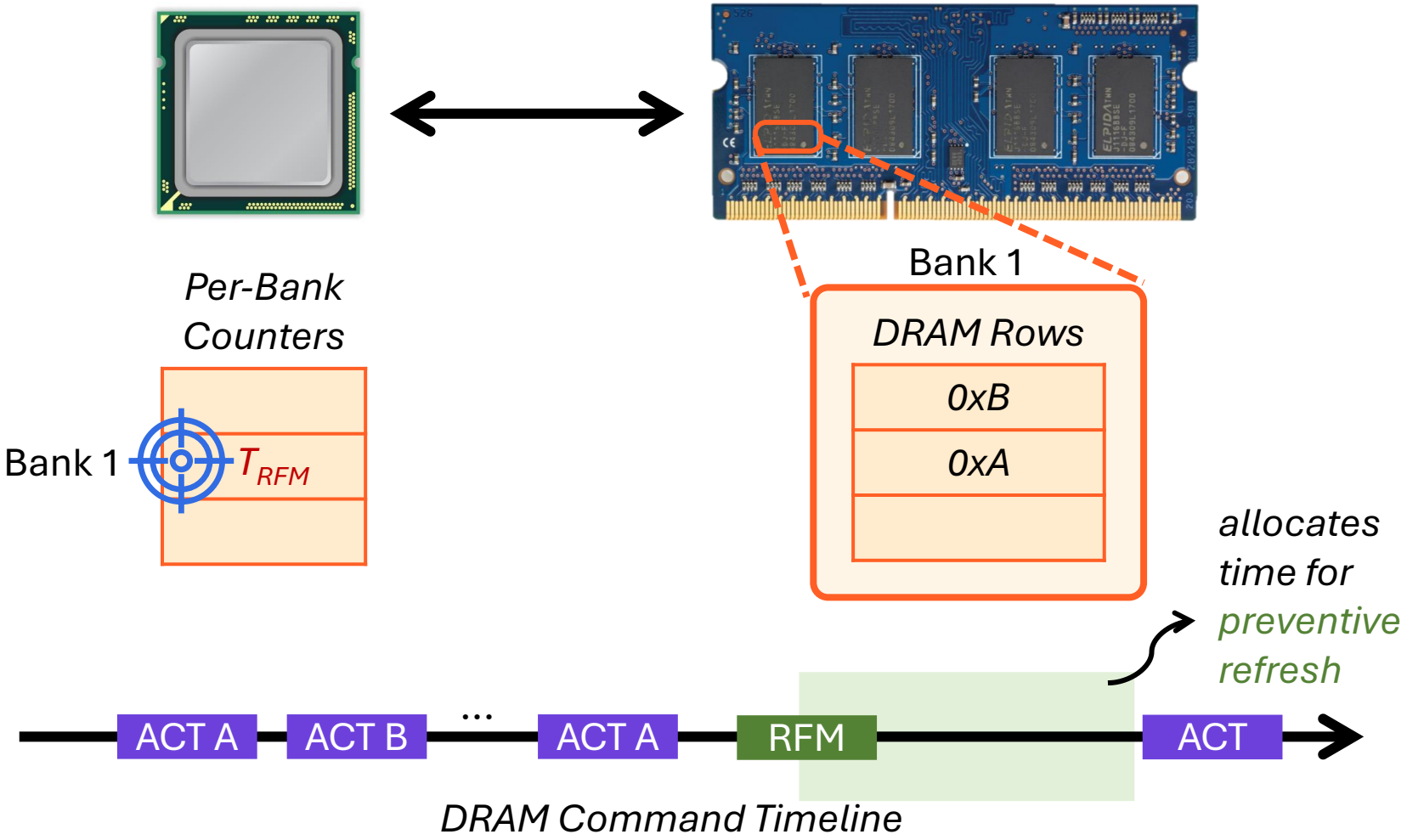
O. Canpolat A. Olgun I. E. Yüksel K. Kanellopoulos
M. Sadrosadati A. G. Yağlıkçı O. Mutlu

ETH zürich SAFARI CISPA HELMHOLTZ CENTER FOR INFORMATION SECURITY

Threat Model

- **Threat Model:** Attack processes can
  - Only share the main memory
  - Flush cache lines with `clflush` and measure time with `rdtsc` instructions
  - Colocate pages in the same DRAM bank by reverse engineering the DRAM address mapping

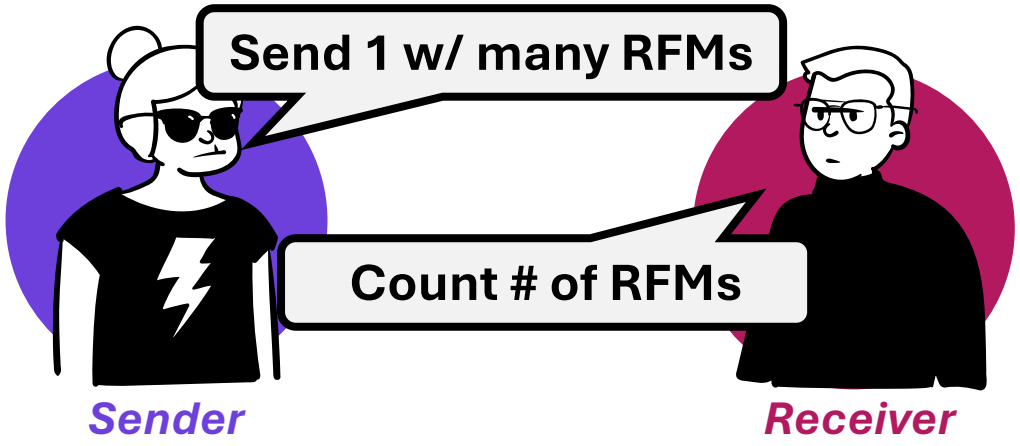
Periodic Refresh Management (PRFM)

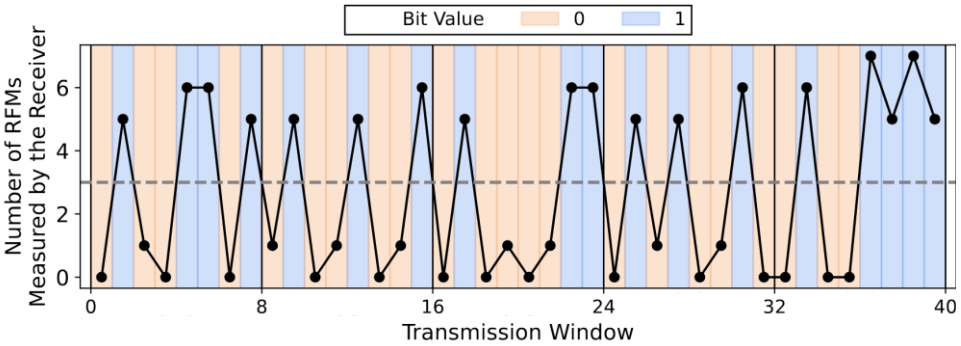


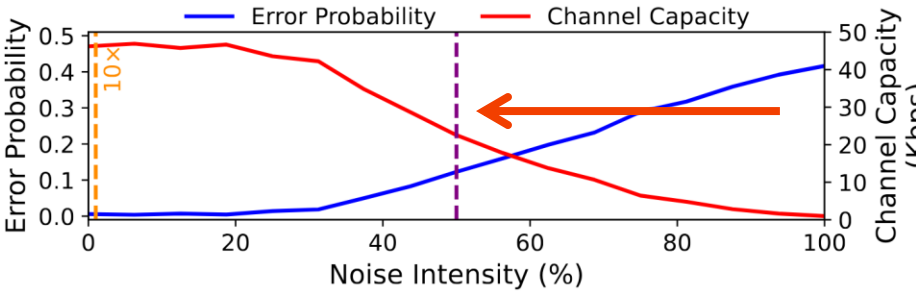
RFM-Based Covert Channel: Attack Overview

Applications can intentionally trigger RFM
and observe RFM-induced latency increases

PRFM is more susceptible to noise due to ***per-bank activation counting***







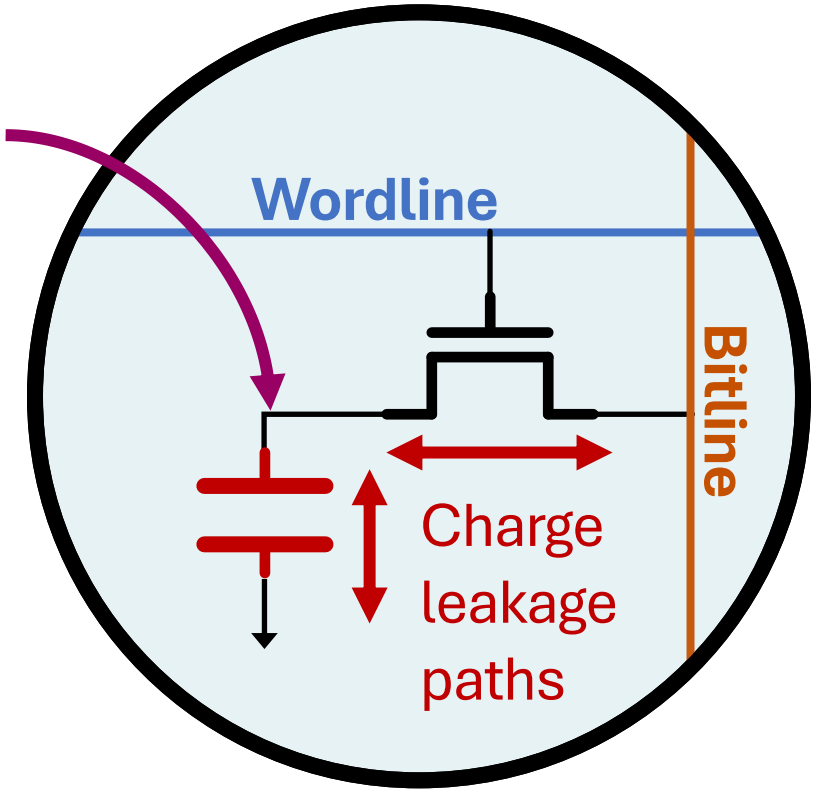
Read Disturbance in Modern Memory Chips

- Prevalent memory technology:
  Dynamic Random Access Memory (DRAM)

- DRAM stores **data** in the form of **electrical charge** on a capacitor

- DRAM **leaks charge** over time and needs **periodic refresh**

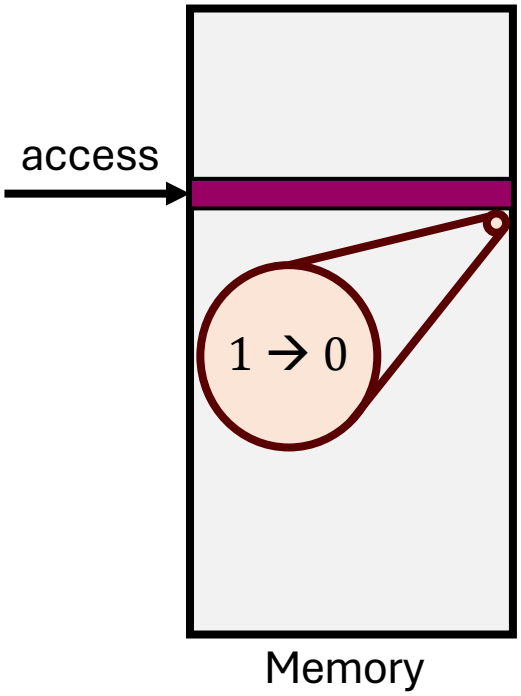
- **DRAM Read Disturbance:**
  Accessing a DRAM cell disturbs other **physically nearby cells** and exacerbates their **charge leakage**



[Yaglikci '24]

Read Disturbance Breaks Memory Isolation

- **Memory Isolation:** A fundamental property for **robustness** (safety, security, and reliability)
- *A memory access **has unintended side effects** on data stored in **other addresses*** due to read disturbance





Data loss or **corruption**



Compromise **application correctness**



Leak private information



Take over a computer

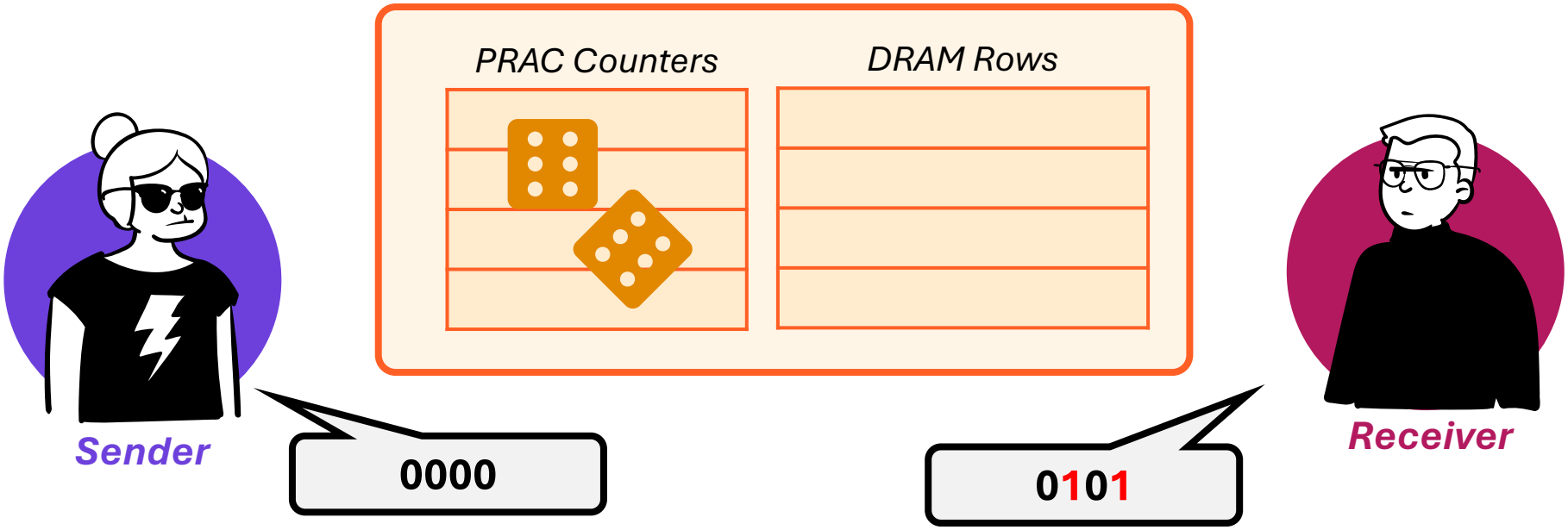
Thus, **mitigating read disturbance bitflips** is **important**

[Yaglikci '24]

How can we **mitigate LeakyHammer**?

Randomly Initialized Activation Counters (RIAC)

- Initializes each activation counter with a random value upon performing a preventive action
  - Randomizes the number of activations needed for a preventive action



RIAC introduces **noise in measurements** and **reduces channel capacity**

Conclusion

Key Observation: RowHammer preventive actions have two key features that enable covert channel and side channel attacks. Preventive actions:

- Result in high memory access latencies observable from userspace applications
- Can be deterministically triggered as they highly depend on memory access patterns

LeakyHammer: A new class of timing attacks that exploit RowHammer defense-induced memory latency differences to *1)* establish covert communication channels between processes and *2)* leak sensitive data

Case Studies: Systematically analyze two latest industry defenses to RowHammer

- Build **two covert channel attacks** to exchange messages between attacker processes
- Demonstrate **a website fingerprinting attack** that can identify visited websites with high accuracy only by observing victim's preventive actions

Mitigating LeakyHammer: Propose and evaluate three countermeasures

- One of our countermeasures completely mitigates LeakyHammer with low performance overhead in near-future DRAM-based systems
- In systems that are highly RowHammer-vulnerable, countermeasures that reduce channel capacity have lower performance overhead

https://github.com/CMU-SAFARI/LeakyHammer

Sensitivity to Number of RFMs per Back-off

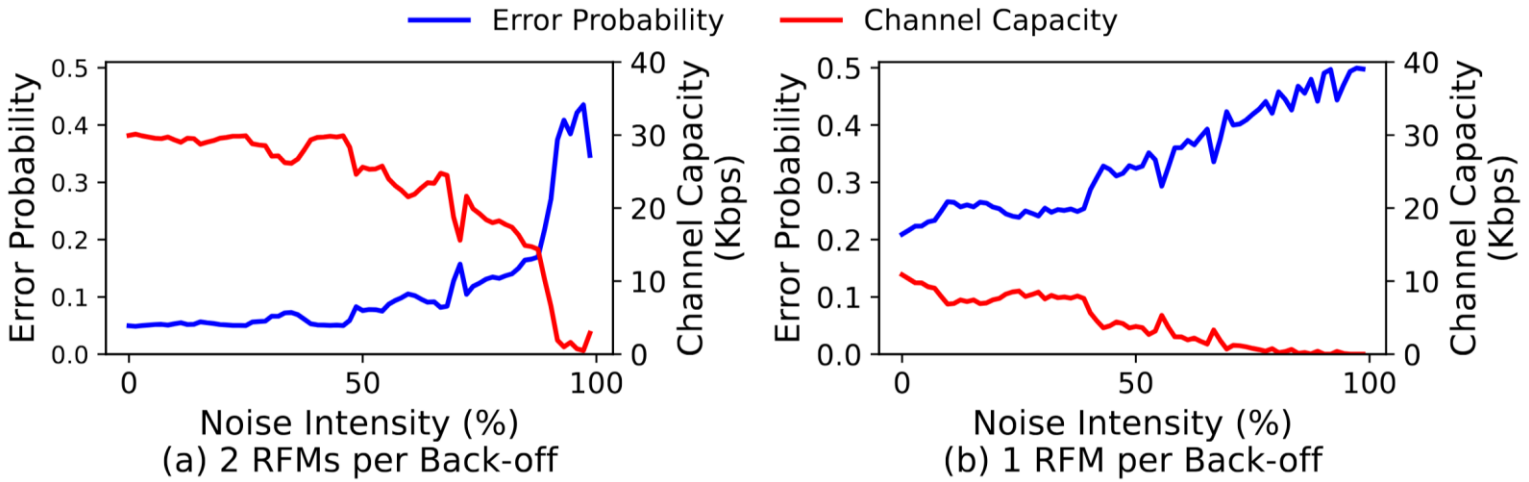
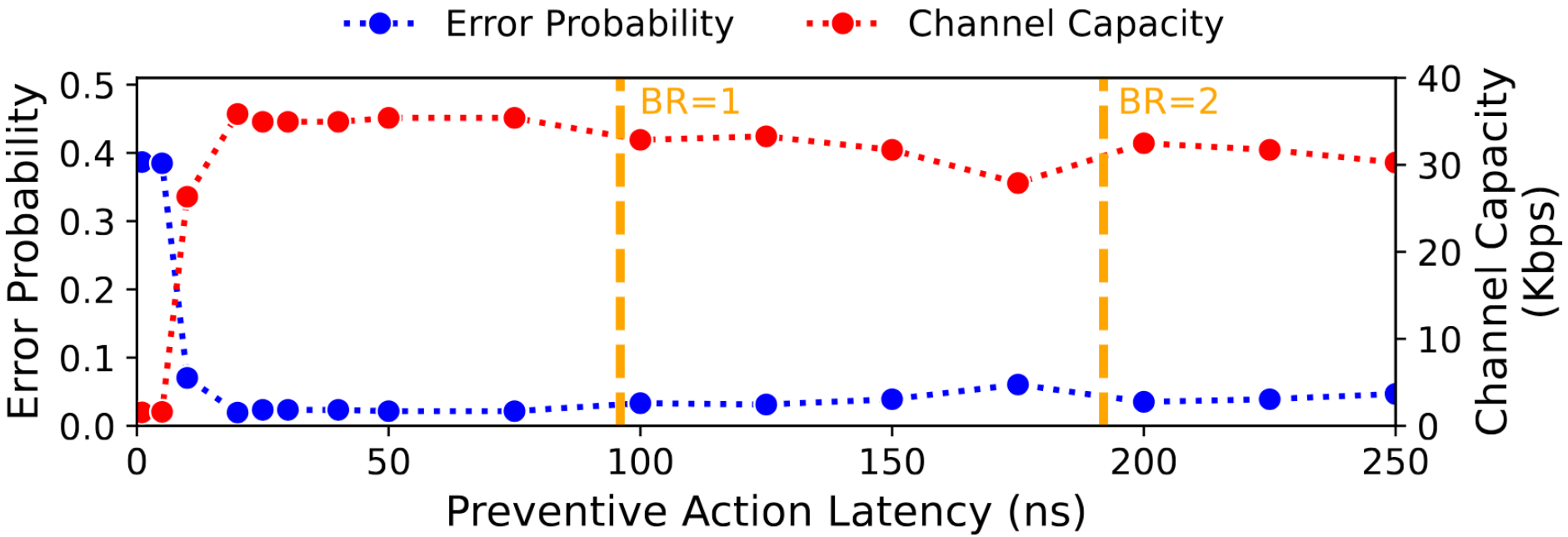


Figure 9: Error probability and channel capacity of Leaky-Hammer with a) 2 RFMs and b) 1 RFM per back-off.

Overlapping back-off and periodic refresh latencies
degrades the channel capacity due to the increased error probability.
However, the attacker can restore the channel capacity by modifying the attack.

Sensitivity to Preventive Action Latency



Reducing the mitigation latency only prevents LeakyHammer for values smaller than 10 ns, which is far below the minimum latency required for preventive refresh

Channel Capacity Metric

We evaluate our covert channels using the channel capacity metric [144]. Channel capacity is the raw bit rate multiplied with $1 - H(e)$, where e is the error probability and $H(e)$ is the binary entropy function. It is calculated as follows.

$$ChannelCapacity = RawBitRate \times (1 - H(e))$$
$$H(e) = -e \log_2(e) - (1 - e) \log_2(1 - e) \quad (1)$$

Latency Measurement Routine

Listing 1: Latency measurement routine.

```
1  // row_ptrs array has two address pointers
2  // located in separate DRAM rows
3  // in the same DRAM bank
4  vector<uint64_t> measure (vector<char*>& row_ptrs) {
5     vector<uint64_t> measured_latency(ITERATIONS, 0);
6     // start timestamp
7     uint64_t start = m5_rpns();
8     for (int i = 0; i < ITERATIONS; i++) {
9         int a = i % row_ptrs.size();
10        auto row_ptr = row_ptrs[a];
11        clflush(row_ptr);
12        // access a target row
13        *(volatile char*)row_ptr;
14        // end timestamp
15        uint64_t end  = m5_rpns();
16        // record the measured latency
17        measured_latency[i] = end-start;
18        start = end;
19    }
20    return measured_latency;
21 }
```

Fingerprinting Routine

Listing 2: Fingerprinting routine collecting memory request latency measurements.

```
1  // row_ptrs array has N address pointers
2  // that point to N different DRAM rows
3  // T = PRAC back-off threshold - 1
4  vector<uint64_t> measure (vector<char*>& row_ptrs) {
5      vector<uint64_t> latency(ITERATIONS*T,0);
6      uint64_t start = m5_rpns();
7      for (int i = 0; i < ITERATIONS; i++) {
8          int a = i % row_ptrs.size();
9          auto row_ptr = row_ptrs[a];
10         for (int j = 0; j < T; j++) {
11             clflush(row_ptr);
12             *(volatile char*)row_ptr;
13             uint64_t end   = m5_rpns();
14             latency[i] = end-start;
15             start = end;
16         }
17     }
18     return latency;
19 }
```

PRAC-Based Covert Channel Capacity: SPEC2017

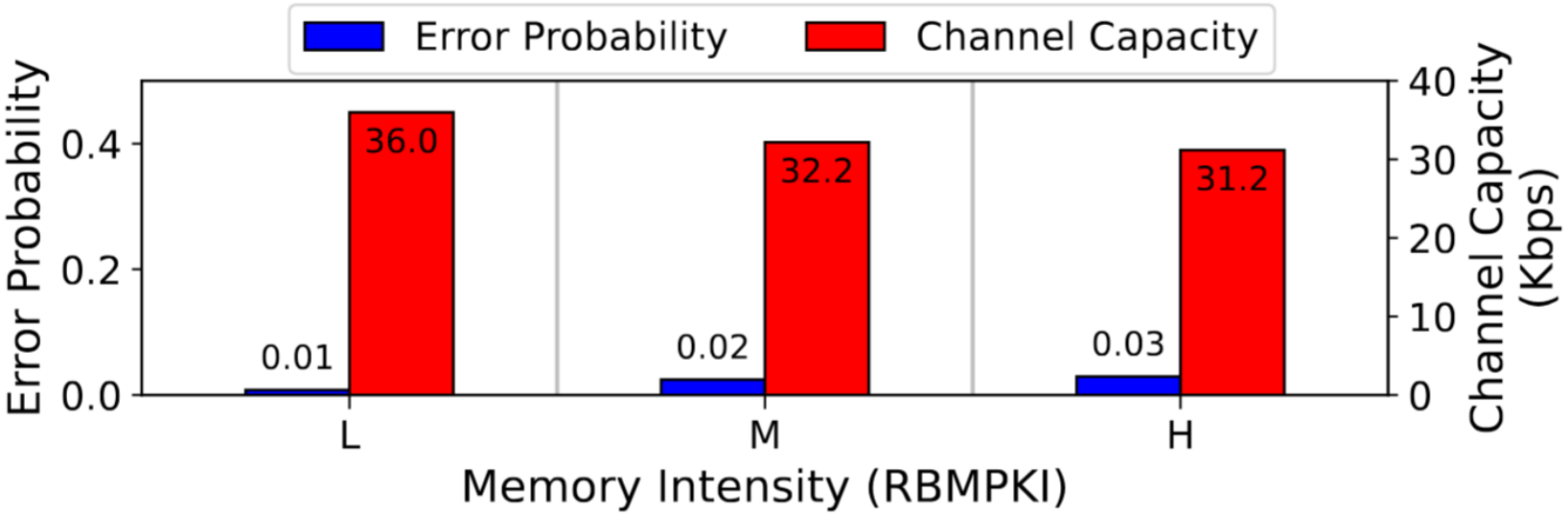


Figure 5: PRAC-based covert channel's capacity and error probability with concurrently running SPEC2017

RFM-Based Covert Channel Capacity: SPEC2017

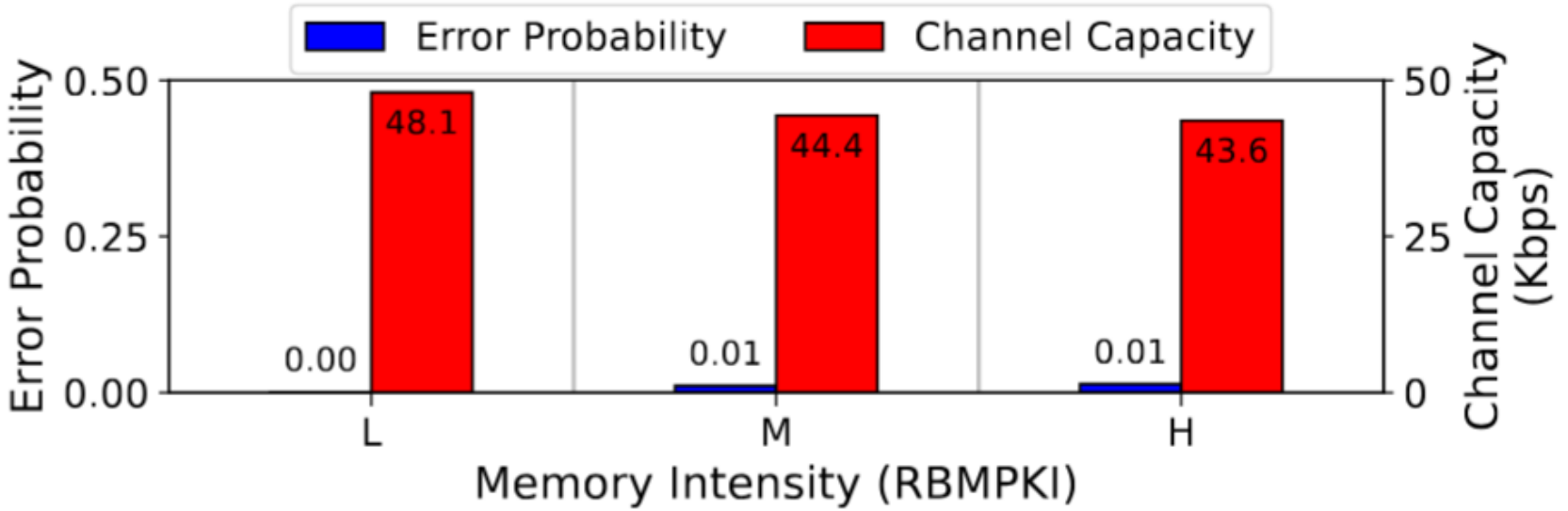


Figure 8: RFM-based covert channel's capacity and error probability with concurrently running SPEC2017

Best-Performing Model: F1, Precision and Recall Scores

Table 2: Website fingerprinting performance employing 10-fold cross-validation: F1 (%), Precision (%), and Recall (%).

Model Name	**F1** Mean (σ)	**Precision** Mean (σ)	**Recall** Mean (σ)
Decision Tree	71.8 (4.2)	74.1 (4.4)	72.4 (4.2)

Impact of Cache Hierarchy and Data Prefetching

Impact of Preventive Action Thresholds

- PRAC back-off threshold of 128 :
  - Low performance overhead while mitigating RowHammer bitflips with low testing overhead
- Covert Channel Capacity:

PRAC Back-off Threshold	Channel Capacity
32	133.4
64	73.8
128	39.0
256	20.0
512	10.2

- RFM-based covert channel capacity shows a similar trend

Information leaked by LeakyHammer and DRAMA

Table 3: Information leaked by LeakyHammer and DRAMA [33]

Colocation Granularity	**Channel / Bank Group**	**Bank**	**Row**
LeakyHammer-PRAC	Victim triggered a preventive action (i.e., memory access pattern)	Victim triggered a preventive action (i.e., memory access pattern)	Number of times the victim has activated the row (i.e., activation count)
LeakyHammer-RFM	Victim triggered a preventive action (i.e., memory access pattern)	Number of times the victim has activated a row in the same bank	Number of times the victim has activated a row in the same bank
DRAMA [33]	N/A	Victim accessed a conflicting row or the same row	Victim accessed a conflicting row or the same row

Comparison Against Concurrent Work (I)

- LeakyHammer is a new class of attacks that exploit RowHammer defense-induced latency differences
  - We evaluate two latest industry defenses
  - We analyze other academic and industry RowHammer defenses

LeakyHammer with Other RowHammer Defenses

- A RowHammer defense-based timing channel is possible when an attacker can both
  - observe a preventive action's overhead
  - intentionally trigger a preventive action

- The channel depends on two properties of the mechanism:
  - Preventive action (observable or overlapped)
  - Trigger mechanism (exact, approximate, random)

Comparison Against Concurrent Work (II)

- LeakyHammer is a new class of attacks that exploit RowHammer defense-induced latency differences
  - We evaluate two latest industry defenses
  - We analyze other academic and industry RowHammer defenses
- We demonstrate a website fingerprinting attack
- There are differences in attack models
  - We assume fewer attacker capabilities (e.g., attacker and victim does not need to strictly synchronize)

Noise Intensity Formula

$$NoiseIntensity = (1 - \frac{SleepDuration - MinSleep}{MaxSleep - MinSleep}) \times 99 + 1 \quad (2)$$